MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR
ET DE LA RECHERCHE SCIENTIFIQUE
UNIVERSITÉ DE TUNIS EL MANAR
INSTITUT SUPÉRIEUR D'INFORMATIQUE

RAPPORT DE STAGE DE FIN D'ÉTUDES

Présenté en vue de l'obtention du

Diplôme National de Licence en Ingénierie des Systèmes Informatiques

Spécialité : Ingénierie des Réseaux et Systèmes

Par

Omar Bounawara

LLM Guided Focused Crawler

Encadrant professionnel : M. Moez Ferchichi

Encadrant académique : M. Moez Ferchichi

Réalisé au sein de l'Institut Supérieur d'Informatique

Année Universitaire : 2025–2026

Table des matières

General Introduction

Exploiting the data available on the Web is today a major concern for many fields, including competitive intelligence, technology watch, and the training of artificial intelligence systems. As the volume of information published online keeps growing, collection methods must be able to identify the most relevant resources efficiently while keeping the cost of exploration under control.

In this context, focused crawling is an approach that steers exploration toward a specific topic rather than seeking exhaustive coverage of the Web. Unlike exhaustive crawling, which is mainly used by general-purpose search engines, this approach concentrates resources on the content most relevant to a given domain [1, 2].

The problem addressed by this project is therefore the following : how can a focused web crawler be designed that exploits pre-existing models to guide exploration, while offering full visibility into the decisions taken and the evolution of the collection process ?

To address this problem, we developed CrawlViz, a focused web crawler designed to automatically explore the Web around a given topic by prioritizing the most relevant resources.

This report is organized into four chapters. The first chapter presents the project's context, the fundamental concepts of web crawling, and the requirements analysis. The second chapter details the theoretical foundations and the design choices adopted. The third chapter describes the system's modeling through the various UML diagrams. The fourth chapter, finally, is devoted to the implementation of CrawlViz and the evaluation of the proposed solution.

1

General Project Framework

Introduction

Designing a focused crawling system requires a prior understanding of the fundamental principles of web crawling as well as the existing approaches for steering exploration toward relevant resources. Before defining a suitable solution, it is therefore necessary to identify the characteristics of the domain, the limitations of traditional methods, and the requirements the system must satisfy.

This chapter first presents the fundamental concepts related to web crawling and the main approaches to focused collection. It then analyzes the positioning of the proposed solution before defining the functional and non-functional requirements that will serve as the basis for the design and implementation phases.

1.1 Technological Context and State of the Art

This section presents the fundamental concepts related to web crawling as well as the main approaches used for focused information collection. It positions the project relative to existing solutions and justifies the choices made for the design of CrawlViz.

1.1.1 Scraping and Crawling

Web scraping refers to the operation of downloading a web page and extracting structured data from it. It is a unitary operation centered on a single resource [3].
Web crawling is a systematic exploration of the web graph : the system starts from one or more initial pages (*seeds*), extracts the links present on each visited page, and continues exploring by following those links. Crawling encompasses scraping as its base operation and adds a layer of discovery and navigation logic on top of it [2].

1.1.2 The Web Graph

The Web can be modeled as a directed graph $G = (V, E)$ [1], where each vertex $v \in V$ represents a page and each directed edge $(u, v) \in E$ represents a hyperlink.
This modeling has direct consequences for the design of a crawler. The graph is **cyclic** : without duplicate handling, a crawler can traverse the same loop of pages indefinitely. The branching factor is **high and variable** [2] : a page may expose only a few links or several hundred. Local connectivity is strong : home pages are often reachable from almost every other page in the same domain.
For example, a crawl starting with a branching factor of 10 and no targeting mechanism reaches 10^4 pages by the fourth depth level alone, the large majority of which may be off-topic.

1.1.3 Existing Approaches to Focused Crawling

Exhaustive Crawling (BFS/DFS)

Classical strategies traverse the graph breadth-first (*BFS*) or depth-first (*DFS*) without evaluating link relevance. They suit general-purpose search engines seeking maximal coverage [2]

but are unsuited to topic-specific use : they visit relevant and off-topic pages indiscriminately, and produce a massive volume of data that must be filtered afterward.

Classifier-Based Focused Crawling

Focused crawling adds an evaluation step before each visit : every candidate link is scored according to its probability of belonging to the target topic. Classical approaches use Naive Bayes or SVM classifiers trained on an annotated corpus [4, 5]. Although accurate, these approaches require a data-preparation phase that makes them difficult to deploy without dedicated expertise.

Intelligent Crawling with Language Models

The rise of large language models (LLMs) has opened a new approach : using a pre-existing LLM to evaluate a link's relevance without prior training [6].
The LLM acts as a semantic judge capable of comparing the meaning of anchor text against a target topic.
The main drawback is the cost and latency of LLM calls. A two-stage filtering architecture is therefore required : a fast, local first filter (NLP) reduces the candidate set, and the LLM is invoked only on the remaining ambiguous cases.

1.1.4 Comparison of Approaches

Table 1.1 compares the main crawling approaches against practical criteria.

Table 1.1: – Comparison of crawling approaches against practical criteria

Criterion	**BFS/DFS**	**Classifier**	**LLM alone**	**NLP+LLM (CrawlViz)**
Training required	No	Yes	No	No
Topical relevance	Low	High	High	High
Cost per link	Very low	Low	High	Moderate
Implementation	Easy	Complex	Moderate	Moderate
Observability	Low	Variable	Variable	High

CrawlViz combines the advantages of semantic approaches (no training required, high topical relevance) with a two-layer filtering architecture that keeps operational cost at a reasonable level.

1.1.5 Positioning of CrawlViz

Building on the analysis of existing approaches, CrawlViz positions itself as a focused-crawling solution built on a multi-level filtering strategy. The proposed approach combines local NLP techniques for fast link evaluation with language models for situations that require deeper semantic analysis.

This combination limits the number of calls made to language models while retaining a high level of relevance when selecting which resources to explore. The solution requires no dedicated training phase, which makes it easier to use across different application domains. Furthermore, unlike many crawling solutions that focus solely on data collection, CrawlViz places particular importance on the observability of the exploration process. The system provides monitoring, logging, and visualization mechanisms that make it possible to understand the crawler's behavior throughout its execution.

1.2 Requirements Analysis and Specification

The limitations identified in existing approaches make it possible to define the functionality expected of a modern focused-crawling system. These requirements form the basis for the design of CrawlViz.

1.2.1 System Actors

The system involves one primary human actor and one external service.

User The user configures crawl sessions, controls their execution, observes their progress, and consults the collected data. The system does not enforce a strict separation between usage profiles.

LLM Service An external service invoked automatically during execution to evaluate the semantic relevance of discovered links.

1.2.2 Functional Requirements

The system's functional requirements are grouped into four main categories.

Crawl Configuration

— Create, modify, and delete collection configurations.
— Define a configuration's parameters through a dedicated interface.
— Customize a configuration according to the user's needs.
— Define one or more starting sources for exploration.
— Configure the rules guiding the discovery of resources to explore.
— Define the method used to evaluate the relevance of discovered resources.
— Configure the information to be collected during exploration.
— Define the stop conditions for a collection session.

Session Management

— Launch a collection session from an existing configuration.

— View the progress of a session.
— Interrupt a session that is currently running.
— Automatically end a session once the configured conditions are met.

Session Observation

— Display indicators that track a session's progress.
— Keep a history of the events that occurred during exploration.
— Provide detailed information on how a session unfolded.
— Allow a session to be analyzed from the recorded information.

Exploration of Results

— Visualize the progress of exploration in real time.
— Identify the status of discovered resources throughout the process.
— Consult information relating to explored resources.
— Consult the collected data in a usable form.
— Access the full content of the collected information.

1.2.3 Non-Functional Requirements

— **Responsiveness :** the interface must remain fluid while the crawl is running.
— **Robustness :** network errors or temporary unavailability of the LLM service must not interrupt the session.
— **Extensibility :** the system must allow the addition of new exploration strategies and new evaluation services.
— **Adaptability :** the system's behavior must be modifiable primarily through configuration.
— **Traceability :** significant events must be recorded and viewable during the session.
— **Respect for third-party resources :** the system must limit its request rate and respect the constraints of target sites [2].

Conclusion

In this chapter, we presented the project's general context along with the problem underlying the focused collection of data on the Web. The study of the fundamental concepts of crawling and of the main existing approaches made it possible to highlight the limitations of traditional solutions and to justify the choices made for the development of CrawlViz.
This analysis also led to the identification of the functional and non-functional requirements that the solution must satisfy. These requirements form the reference framework for the architectural choices and design decisions detailed in the next chapter.

2

Algorithmic Design Principles and Software Architecture

Introduction

In this chapter, we present the algorithmic choices behind the CrawlViz system, addressing the main challenges involved in designing a focused crawler :

— guiding exploration without prior knowledge of the Web's structure ;
— structuring lexical filters for selecting relevant content ;
— the trade-off between processing efficiency and evaluation cost ;
— integrating a language model without creating a bottleneck in the system ;
— handling network constraints and the mechanisms that mitigate them.

2.1 Semantic Representation of the Target Topic

2.1.1 The Problem of Unfocused Crawling

In the case of Wikipedia, a BFS starting on the "Black Hole" page quickly drifts into science-fiction or video-game pages. These topics are heavily interlinked within the link ecosystem

and are often off-topic when one is looking for scientific content.
Indeed, BFS relies solely on the structure of the link graph, with no notion of semantic similarity [1, 2]. A mechanism is therefore needed to approximate this semantic similarity.

2.1.2 Vector Encoding of Text

Using embedding models, texts are projected into a fixed-dimensional vector space, making it possible to measure their similarity via metrics such as cosine distance [7].
These models transform texts into fixed-dimensional representations (typically 384 or 768 dimensions), allowing the semantic similarity between two pieces of content to be computed without lexical comparison.

2.1.3 Multi-Facet Representation of the Topic

The problem that arises is the following :
At the outset, the vector basis is empty : the system does not yet have enough information about the target.
Indeed, all it has are the initial pages, the target pages, and a short (configured) description of the objective.
This produces overly weak signals ; more precisely, the scoring mechanism ends up behaving as a purely lexical similarity function.
One solution is to use conceptual expansions, i.e., to enrich the space at the outset with LLM-generated enriched conceptual descriptions of the topic. These are then encoded into vectors and form the initial scoring basis. This mechanism constitutes a cold-start embedding-bootstrapping strategy.
The relevance of a candidate link is computed as the maximum cosine similarity between the encoding of the link and one of the basis vectors :

$$\text{score\_nlp}(c) = \max_i \frac{E(c) \cdot \mathbf{b}_i}{\|E(c)\| \, \|\mathbf{b}_i\|} \tag{2.1}$$

where $E(c)$ is the encoding vector of candidate link c, and $\mathbf{b}_i$ is the i^{th} vector of the semantic basis.

2.2 Link Modeling and Signal Extraction

2.2.1 Multi-Source Encoding of Links

A large part of focused crawling relies on estimating the relevance of a link before it is actually visited.

To this end, the system exploits as much information as is available at the level of the parent page, in order to build an enriched representation of the candidate link.

For example :

```
<li>
institut superieur informatique ISI, <a href="https://isi.rnu.tn">visit</a>
</li>
```

From this kind of structure, several sources of information can be extracted :

- the link's anchor text, i.e., the clickable text contained in the `<a>` tag, for example : "visit" ;
- the target URL (`href`), for example : https ://isi.rnu.tn ;
- the local textual context, extracted from the nearest parent HTML node, for example : "institut superieur informatique ISI."

This multi-source representation provides a more robust signal for relevance evaluation. When one piece of information is weakly discriminative (for example, a generic anchor text such as "read more," or an empty context), the other components of the representation compensate for that weakness.

2.3 Two-Stage Scoring Architecture

2.3.1 The Problem of Evaluation Cost

Systematically evaluating every link with an LLM is impractical :

- an explosion in the number of requests (each page contains many links) ;
- high token cost and significant latency ;
- a bottleneck in the exploration pipeline.

A two-stage architecture is therefore necessary.

2.3.2 Cascade Architecture

A cascade architecture is commonly used in modern information-retrieval systems, where a fast first filter reduces the search space before a more expensive evaluation [1].

The overall priority function is defined as follows :

$$P(n) = \lambda_1 S_{\text{NLP}}(n) + \lambda_2 S_{\text{LLM}}(n), \quad \lambda_1, \lambda_2 \in \mathbb{R} \tag{2.2}$$

Layer 1 – Local NLP Filtering

The system encodes the representation of each link (anchor text, context, URL) using an embedding model. These representations are then compared against a local semantic basis, allowing similarity to be computed in a few milliseconds.

The NLP score is derived from several scalar signals extracted from this vector space :

- semantic similarity with the target topic ;
- the link's ability to explore still poorly-covered areas ;
- novelty relative to already-visited documents ;
- contextual coherence between the link and its source page ;
- lexical overlap with the topic's keywords ;
- local density in the embedding space ;
- the estimated distance between the link and already-identified clusters.

These signals are aggregated through a weighted combination, whose weights are set dynamically according to the selected crawling strategy (`AGGRESSIVE`, `EXPLORATION`, etc.).

Figure 2.1: – Prioritization strategies used in the system.

Bucketing of Candidates

After the NLP score is computed, links are sorted into three groups against two configurable thresholds (H) (`HIGH`) and (L) (`LOW`) :

- score ($< L$) : these are the links that are generally far from the target within the basis ;
- score $\in [L, H]$: ambiguous links for which the NLP score does not allow a reliable priority decision ;
- score ($> H$) : links strongly consistent with the target within the basis ;

From these three groups, the links to be passed on to the LLM are collected :

- among the low scores, the system randomly samples against a budget in order to avoid bias toward already-known regions, more precisely to avoid the problem of converging on a local optimum ; the rest is then ignored by the system.
- among the high scores, sampling is performed both through a top-(K) approach and randomly against a budget, in order to reinforce reliable semantic regions and increase precision. In this case, the rest is excluded from LLM evaluation ; priority is then computed solely from the strategy-dependent NLP signals.
- the entirety of the links in the intermediate group.

Layer 2 – LLM Evaluation

The selected links are passed to the language model. The resulting score is then combined with the NLP signals to compute the final priority of the new node.

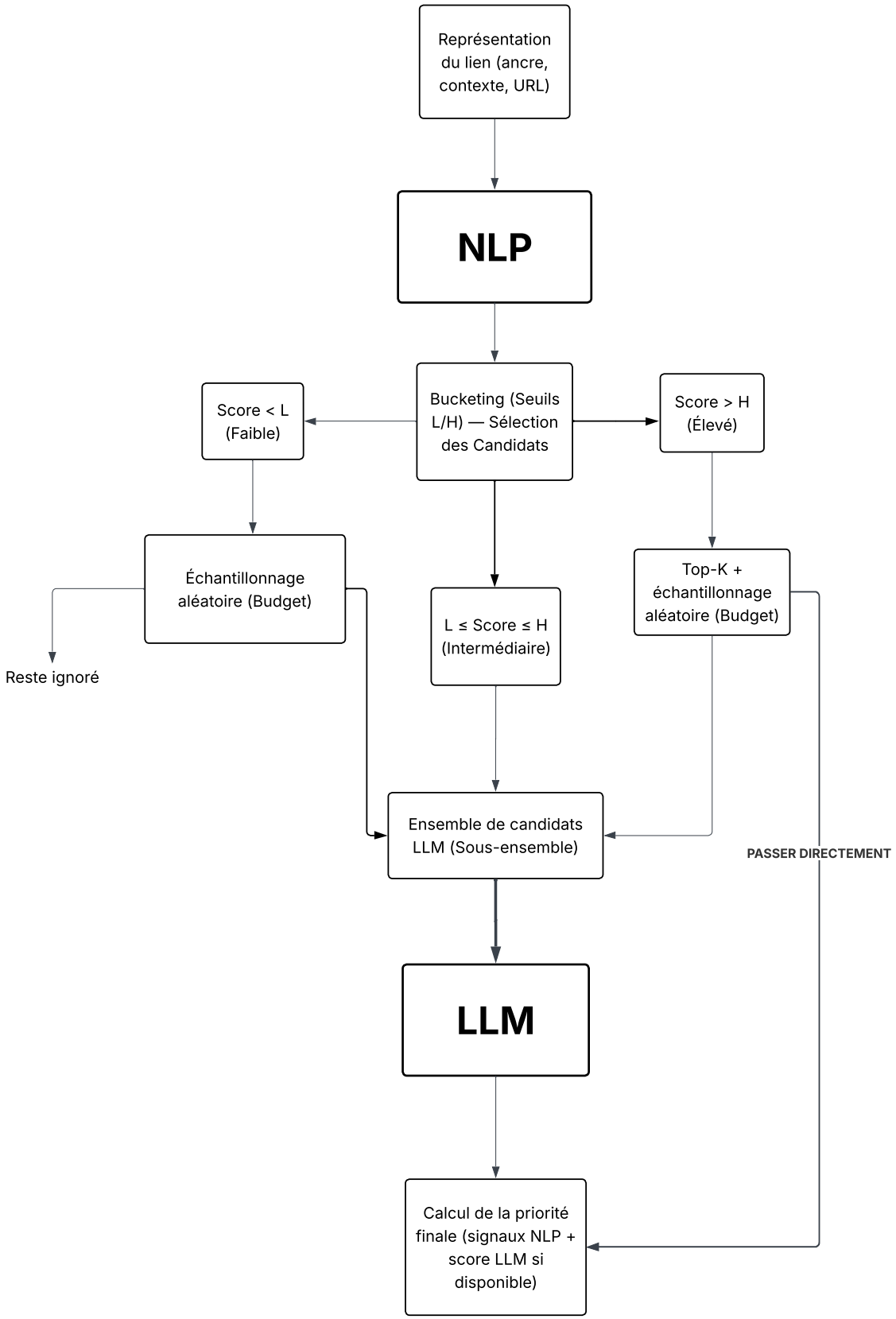

Figure 2.2: – Cascade scoring pipeline.

2.3.3 Benefit of the Hybrid Architecture

This architecture substantially reduces computational cost while retaining a high selection quality, thanks to a clear separation between fast filtering and deep semantic evaluation.

2.4 Dynamic Enrichment of the Semantic Basis

The LLM evaluates the links[6] selected by the NLP layer. It produces :

- a relevance score ;
- a semantic category ;
- conceptual expansions.

These expansions are periodically integrated into the basis, which progressively improves scoring quality and enables their persistence for reuse across other sessions.

2.5 Resilience Strategies

In a crawling system, network operations are the main source of uncertainty.
By nature, HTTP response time ranges from a few milliseconds to several seconds ; the system can also encounter servers that do not respond at all.
There is also the problem of rate limiting by sites : a large volume of requests can trigger a "TOO MANY REQUESTS" block.
For transient errors – timeouts, momentary overloads, rate limits – an automatic retry strategy is implemented using an increasing delay (*exponential backoff*) [8].
After each failure, the waiting time is multiplied by an exponential factor.
More formally :

$$\Delta t_k = \min(\Delta t_0 \cdot \beta^k, \Delta t_{\max}) + \text{jitter}, \quad \text{with jitter} \sim \mathcal{U}(0, \text{jitter}_{\max}) \tag{2.3}$$

Jitter desynchronizes parallel attempts in order to avoid the *thundering herd* effect (several repeated requests firing at the same time).
This approach helps avoid overloading the server while still guaranteeing an automatic recovery.

2.6 Asynchronous Architecture and Processing Pipeline

2.6.1 Justification for Asynchrony

Crawling is dominated by I/O operations. Sequential execution leads to massive under-utilization of resources.

Tests show a significant gain between sequential and concurrent execution.

Approach	**Nodes**	**Links discovered**	**Time**
Sequential	560	65000	600+ s
Concurrent	560	65000	15 s

Table 2.1: – Comparison of performance by approach

We observe that the concurrent approach is significantly more efficient, because the sequential approach blocks the entire system during every wait.

By contrast, the asynchronous mode can handle dozens of operations without creating as many threads, which limits memory consumption and reduces synchronization complexity [9].

2.6.2 Pipeline and Pub/Sub Model

The system relies on a decoupled architecture :

The system explicitly decouples production and consumption. This is achieved through stateless pipelines.

The producer finishes its task and immediately resumes its cycle, without waiting for the consumer. The consumer receives the result asynchronously.

The system uses a *pub/sub* model, which guarantees ease of extensibility and modification via subscriptions [9].

In addition, each pipeline can have a variable number of workers, depending on the system's objectives. For example :

— a single worker is enough for the database ;

— several workers can be used for request processing when volume increases.

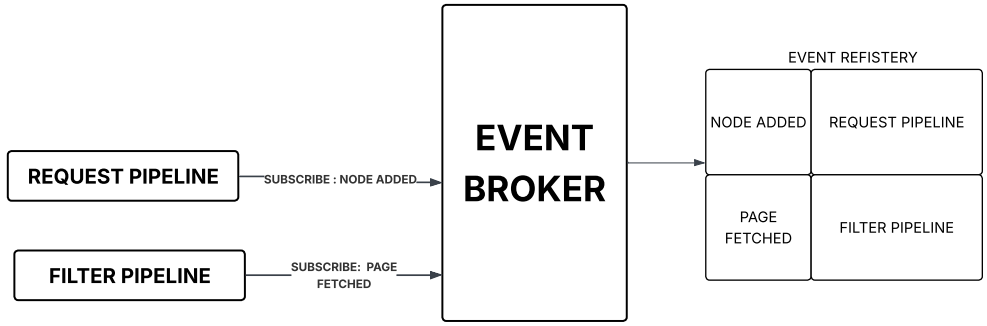

Figure 2.3: – Subscription mechanism of the pipelines to the system's events.

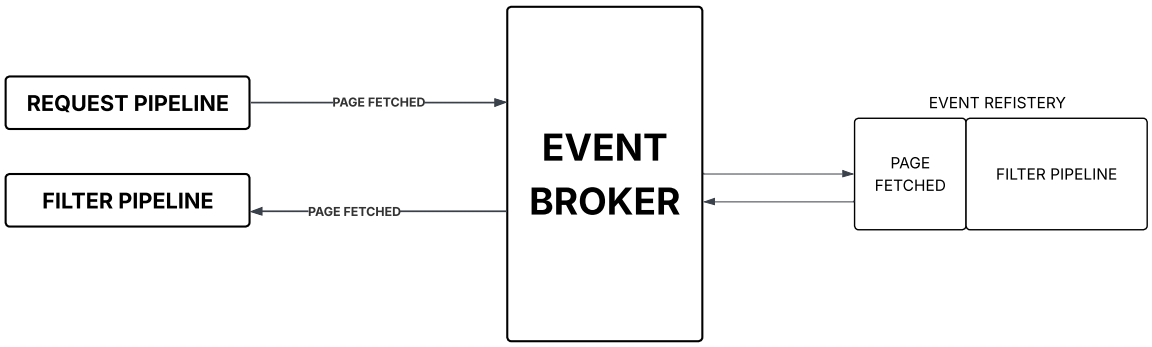

Figure 2.4: – Routing of events via the system's central pub/sub bus.

2.6.3 Simultaneous Triggering and Synchronization Mechanism

This architecture also guarantees the expected behavior of the system, in particular the resolution of *out-of-order* issues. These mainly result from a race condition on the shared priority queue, manifesting as disordered execution when nodes are selected. This phenomenon was observed during tests of an architecture coupling the requests pipeline and the scoring pipeline.

Table 2.2: – Illustration of the concurrency problem with a shared priority queue

Time	**Global queue**	**Worker 1**	**Worker 2**
t0	–	A(90) → C(100) D(80)	B (60)
t1	C(100), D(80)	A (finished → C taken)	B (still processing)
t2	D(80)	C (100)	B (finished → flow continues)
t3	...	...	D(80)

In this case, among A's links, a new node C is created with priority 100 and D with priority 80. However, the system processes B before the others, because once a node is chosen from the frontier, it is processed to completion.
CrawlViz's architecture resolves this problem by separating the scoring and requests flows : the requests pipeline downloads pages, which are then marked as ready ; the scoring pipeline chooses the node to process and only considers nodes that are already ready.
This guarantees that the system consistently selects the node with the best priority available at any given instant.

Table 2.3: – Transition with explicit separation of workers (2 fetch + 1 scoring)

State	**Fetch Worker 1**	**Fetch Worker 2**	**Scoring Worker**	**Scoring queue**
t0	A(90)	B(60)	C(100) D(70)	B(60)
t1	C(100)	D(70)	C(100)	D(70) B(60)
t2	...	...	D(70)	B(60)
t3	...	...	B(60)	...

2.7 Causal Traceability and Replayability

2.7.1 The Log as a Causal Record

Significant transitions in the system (node created, link scored, etc.) produce an event. These events are routed to the components via a central component, which forwards them to logging modules.

These are separated into two tiers :

Standard logging (`Logging`) : handles the events of a node's lifecycle (creation, filtering, etc.). Each entry contains the node ID, the worker ID, and the state concerned.

Detailed trace (`Debugging`) is an optional component that can capture a wider set of events :

— NLP inputs/outputs ;

— network request sending and receiving.

Together, these two mechanisms build a persistent session history. This trace makes it possible to reconstruct the exploration trajectory and to analyze problems.

Nature of Replayability. The replayability of the CrawlViz system is a projection derived from the event stream. One component maintains the graph's state and applies changes as they arrive. This property is exploited : the graph's state at a given instant t is reconstructed from an empty initial state by applying the sequence of events up to t [9][10].

Formally, for the prefix of the event stream up to t :

$$S_t = \text{project}(S_{t-1}, e_t), \quad S_0 = \emptyset \tag{2.4}$$

where $\{e_1, e_2, \ldots, e_n\}$ denotes the ordered sequence of the system's events.

2.7.2 States in a Node's Lifecycle

The logical states of a node are :

`CREATED` The node exists in storage and is inserted into both the scoring and requests queues with its URL and computed priority. No content is available yet.

`FETCHED` The HTTP response has been received and the page's raw content is stored on the node.

`FILTERED` The filtering component has applied deduplication to the links and elements extracted from the node. This state means that the links found on the node were filtered, *not* that the node itself was rejected from the graph.

`SCORED` The extracted links have been evaluated by the NLP pipeline and, for the selected subset, by the LLM. Each link now carries a relevance score.

`EXPANDED` Priority calculation is complete for the scored links. New child nodes have been created in storage and inserted into the fetching queues. The node has finished contributing to the crawl's progress.

2.7.3 Separation Between Execution and Observation

In the CrawlViz system, two execution planes with different responsibilities and requirements are distinguished :

— **Control plane :** it groups together all operator-system interactions ; this plane is independent of any single session's execution and remains permanently available.

— **Data plane :** it corresponds to the stream of information produced by the currently running session.

2.8 Declarative Blueprint Configuration

2.8.1 Principle

Inspired by domain-specific languages, a *blueprint* is a configuration document that declaratively describes the full set of parameters needed to execute a crawl operation.
It notably groups together :

— the seed pages

— the target pages as well as the stop conditions

— the scoring and priority-calculation strategies

— the semantic-enrichment logic

The system's entire execution logic is implemented once, in the exploration engine. The *blueprint* therefore does not define procedural behavior, only the configuration of that behavior. This approach allows the same configuration to be reused across an arbitrary number of sessions.

2.8.2 ETL Pipeline

The extraction subsystem follows the ETL (Extraction–Transformation–Loading) principle. For each visited page :

1. **Extraction** : the XPath/CSS selectors defined in the blueprint are applied to the HTML in order to extract the target fields (title, paragraphs, etc.).
2. **Transformation** : the extracted values are normalized according to the transformation rules defined in the blueprint. The process is applied as a composition of successive functions.
3. **Loading** : the transformed data is persisted to the database, with a duplicate-detection mechanism based on a content fingerprint.

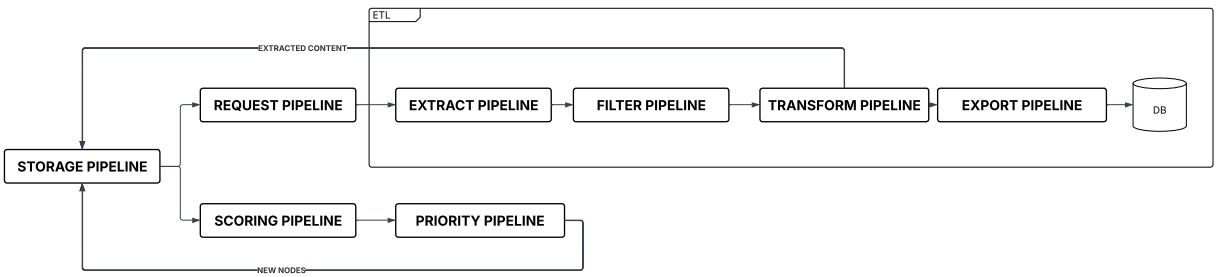

Figure 2.5: – Logical ordering of the crawler's execution pipelines.

Conclusion

This chapter presented CrawlViz's main design choices. The chosen approach combines NLP filtering, evaluation by a language model, and a cascade scoring architecture in order to steer exploration efficiently while keeping processing costs under control.
The asynchronous event-driven architecture ensures the system's modularity, robustness, and traceability. The next chapter presents the concrete implementation of these choices through the technologies used, the implemented architecture, and the system's evaluation.

3

Design

Introduction

This chapter presents CrawlViz's overall design, detailing its internal organization, its main components, and their interactions.

The architecture rests on an asynchronous, event-driven model structured into pipelines, making it possible to efficiently process a large stream of requests while guaranteeing modularity, extensibility, and observability.

The goal is to describe the design choices that ensure separation of concerns, system robustness, and the ability to adapt to variable load.

3.1 Use Cases

The figure below presents a global view of CrawlViz's use cases, identifying the main actors as well as the major functionality offered by the platform.

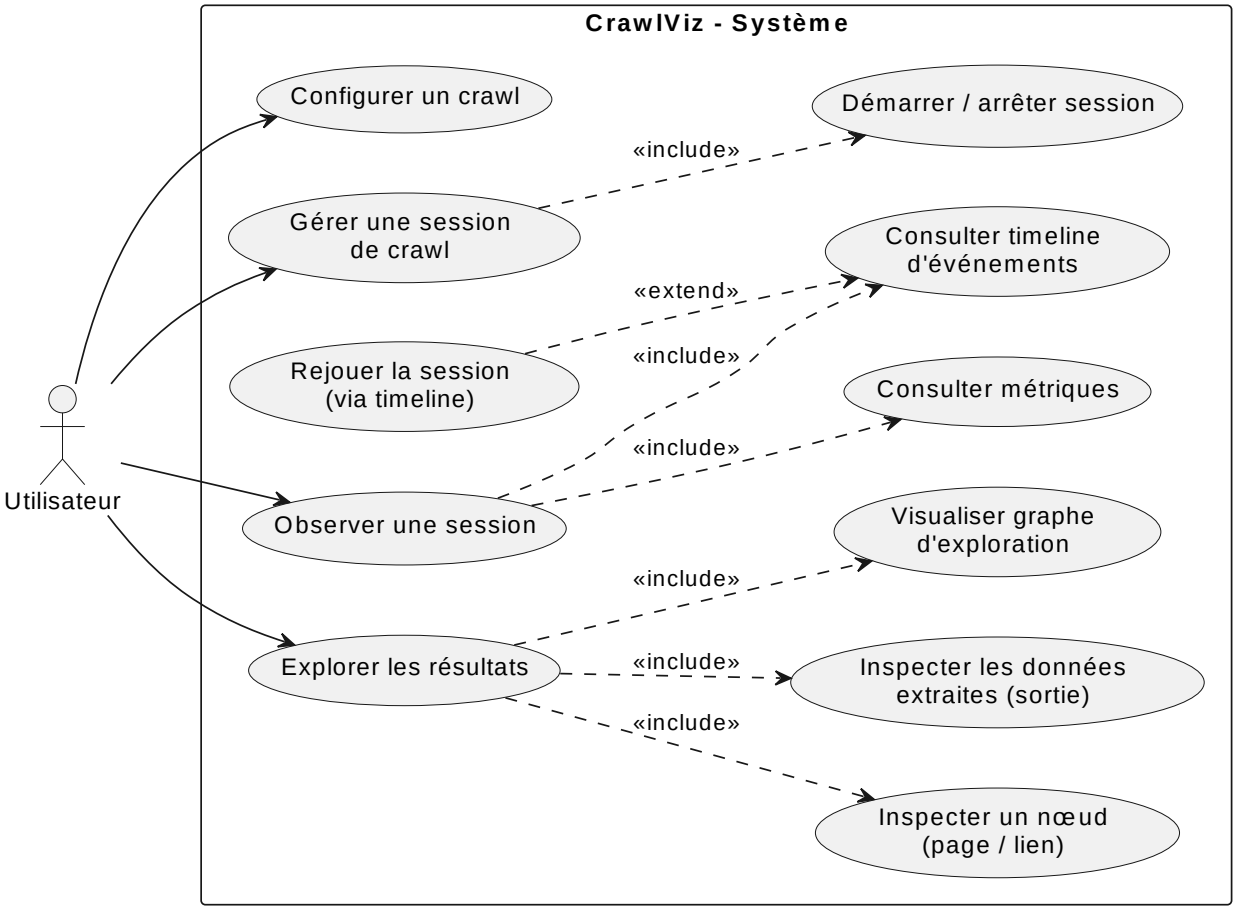

Figure 3.1: – Global use-case diagram

Use cases are grouped according to the system's main functional areas.

3.1.1 Configuring a Crawl

This use case details the process of configuring a crawl session, including the definition of exploration parameters, scoring strategies, and extraction rules.

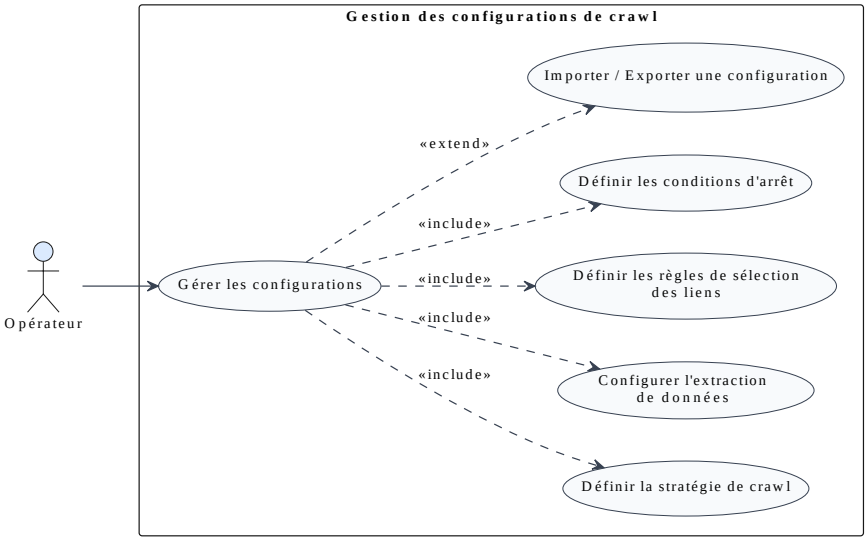

Figure 3.2: – Configuration use case

Actor	User
Preconditions	The system is accessible and no session is active.
Main scenario	The user creates or modifies a configuration by defining the starting pages... The configuration can be edited through a structured interface or directly as a JSON file.
Variants	Loading and modifying an existing configuration.
Exceptions	The system refuses to save the configuration if required parameters are invalid or missing.
Postconditions	The configuration is saved and available for a future session.

Table 3.1: – Description of the use case : configuration

3.1.2 Managing a Crawl Session

This diagram illustrates the interactions involved in managing a crawl session's lifecycle, from launch to shutdown, along with the associated control actions.

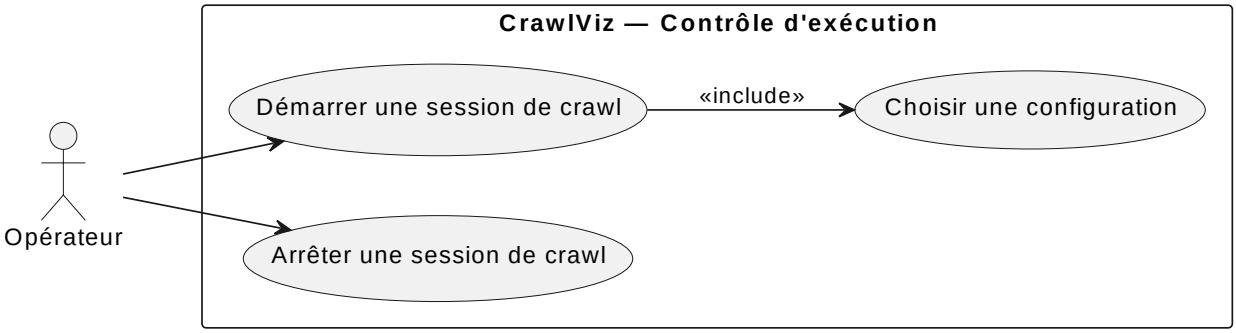

Figure 3.3: – Session-management use case

Actor	User
Preconditions	At least one configuration is available and no session is active.
Main scenario	The user selects a configuration and then starts a session. The system runs the crawl and exposes its current state during execution. The session stops automatically once a stop condition is met.
Variants	The user manually stops a running session.
Postconditions	The session has ended and the collected data is retained.

Table 3.2: – Description of the use case : session management

3.1.3 Observing a Session

This use case describes the observation and monitoring mechanisms for tracking the state of a session, running or finished, as well as access to its execution history.

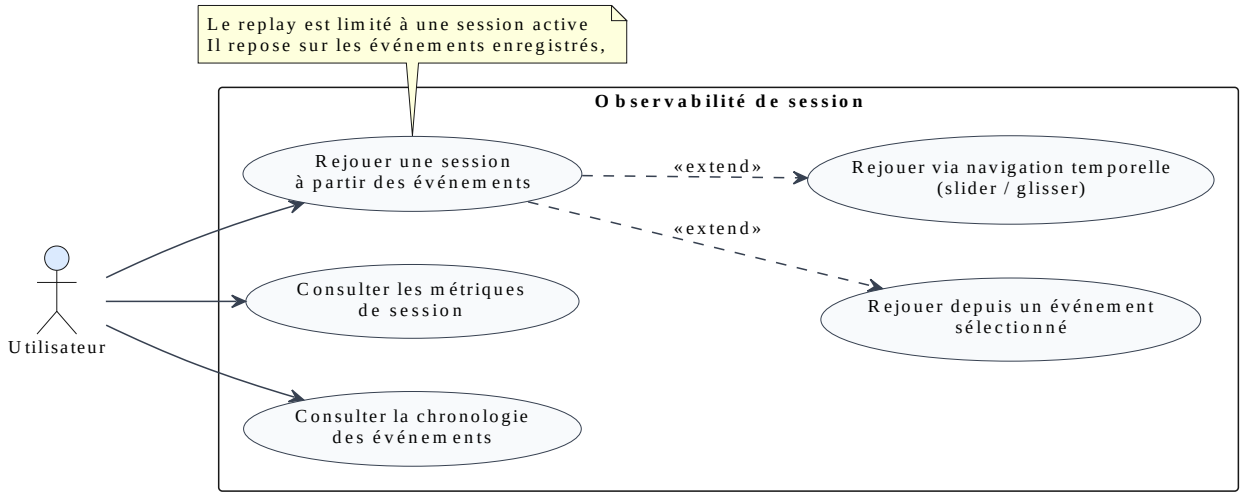

Figure 3.4: – Observation use case

Actor	User
Preconditions	A session is running or has recently finished.
Main scenario	The user consults the aggregated indicators, the timeline of events, and the session's execution log.
Variants	The user enables inspection mode and browses the event history to reconstruct the session's evolution and that of the exploration graph.
Postconditions	The user has a detailed view of how the session unfolded.

Table 3.3: – Description of the use case : observation

3.1.4 Exploring Collected Data

This diagram presents the functionality for exploring collected data, allowing analysis and consultation of the results extracted during crawl sessions.

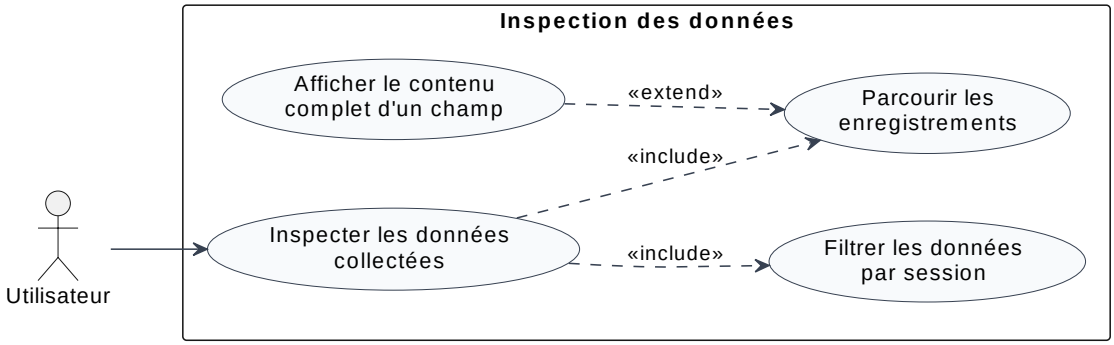

Figure 3.5: – Data-exploration use case

Actor	User
Preconditions	A session has produced usable data.
Main scenario	The user consults the tabular view of extracted data, applies filters by session, and browses the available records. They can select a field to display its full content.
Postconditions	The collected data is accessible in a usable and navigable form.

Table 3.4: – Description of the use case : data exploration

3.2 Global View

The figure below presents an overview of CrawlViz's overall architecture, illustrating the system's main components and their interactions.

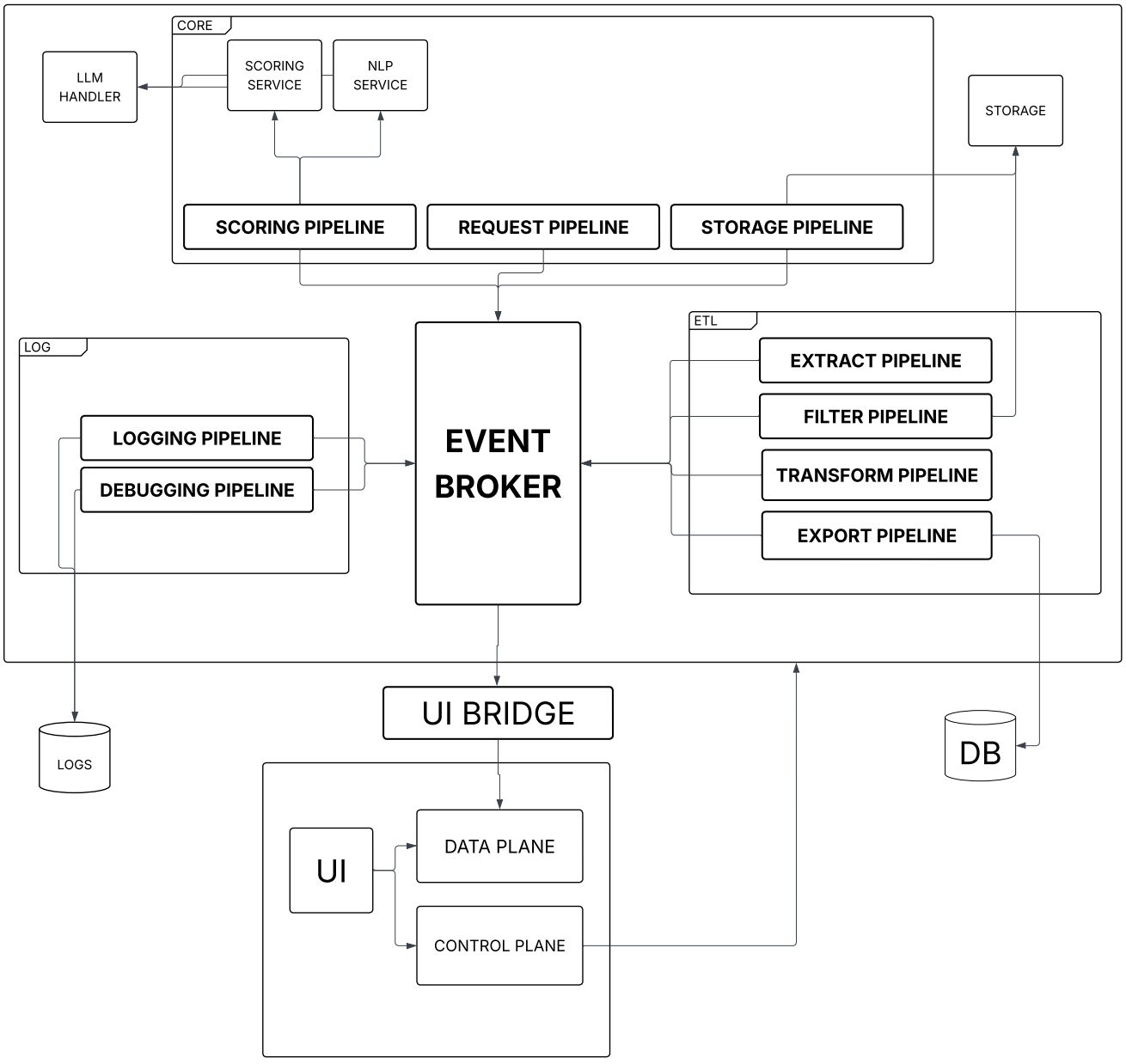

Figure 3.6: – Global view of the CrawlViz system architecture.

The key elements of the architecture can be summarized as follows :

- **Broker and event-driven architecture :** The broker is the core of the system and orchestrates all exchanges through a *pub/sub* model. Each component acts as a producer and/or consumer of typed events, ensuring strong decoupling and natural extensibility of the architecture.
- **User interface (UI bridge) :** The UI bridge is a plain subscriber to the system's events, on the same footing as the other pipelines. It translates internal events into formats usable for visualization.
- **Separation of planes :** The architecture clearly distinguishes a data plane from a control plane. The data plane is exposed read-only through the UI bridge, while the control plane manages execution and supervision of the system, following a logic close to the CQRS (*Command Query Responsibility Segregation*) paradigm.
- **State and storage management :** The system distinguishes two complementary levels. An **in-memory storage object** represents the crawler's dynamic state (nodes, links, execution metadata). This object is the central structure of execution and is exclusively manipulated by the **storage pipeline**, which handles the creation and update of internal entities (nodes, items, relations).

  In parallel, an **export database** is dedicated solely to the final persistence of results. Only the **export pipeline** is allowed to write to this database.
- **NLP/LLM services :** The NLP and LLM components are encapsulated as services. The scoring pipeline accesses them through simple interfaces, without handling the internal complexity of the models, which reinforces the separation of concerns.
- **Logging and debugging :** The logging pipeline is active by default to guarantee full traceability of the system. The debugging pipeline can be disabled to reduce overhead in a production environment.
- **System extensibility :** The architecture allows new components to be added simply by connecting them to the broker through typed events, without modifying the system's core.

The system is thus designed as an event-driven architecture with centralized state, where all mutations pass through explicit, traceable pipelines.

3.3 Class Diagrams

3.3.1 Domain Model

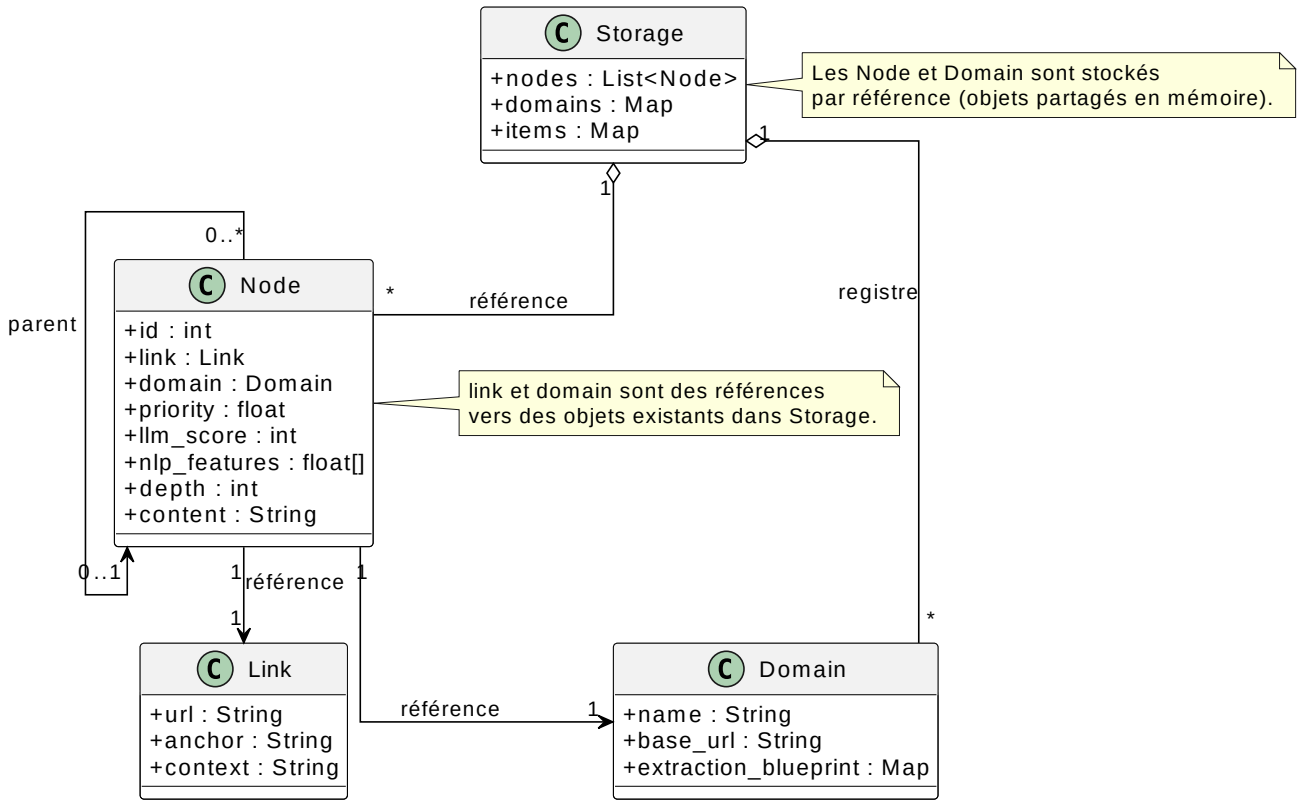

Figure 3.7: – Class diagram of the crawler's domain model

The domain model rests on a central invariant : in-memory storage is the single source of truth. All objects manipulated by the pipelines (nodes, links, domains, extracted items) are references into that central state, with no local persistent duplication or copying.

— **Link (intermediate object)** : a temporary structure representing a navigation candidate used for scoring. After evaluation, it is materialized into a *Node*,

— **Node** : the main entity of the exploration graph. It groups execution metadata (depth, scores, etc.).

— **Domain / Items** : entities persisted in in-memory storage and accessible only by reference, guaranteeing the overall consistency of the state.

3.3.2 NLP Component

This diagram details the internal architecture of the NLP component, responsible for computing similarity signals and the semantic evaluation of links.

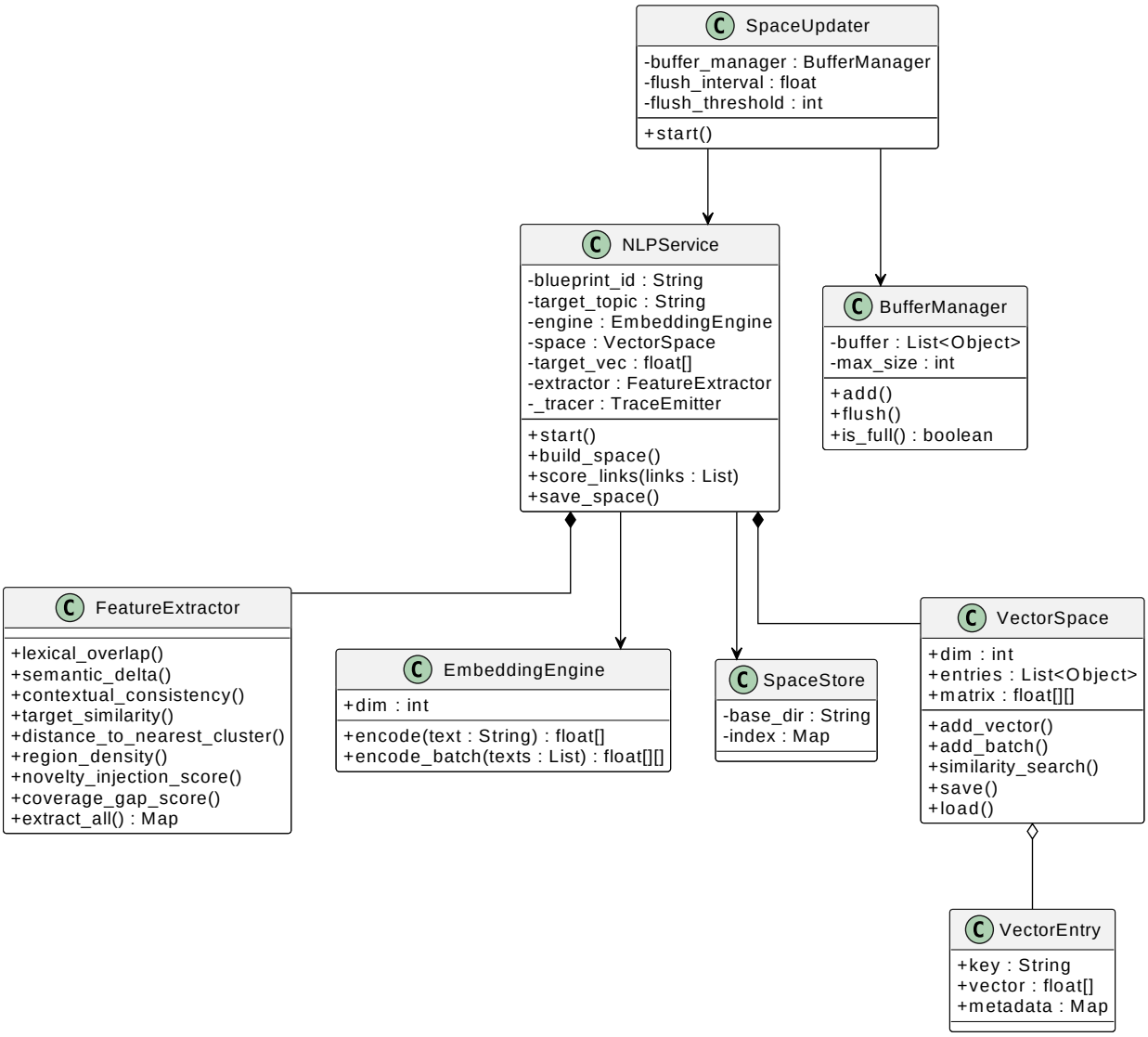

Figure 3.8: – Class diagram of the NLP component

The NLP component is structured around two main entities : the *NLP Service*, used in read mode by the pipelines, and the *Space Updater*, responsible for the progressive evolution of the vector space.

— **NLP Service** : the central interface used by the scoring pipeline. It encapsulates the embedding engine, the feature extractors, and the management of the vector space. It provides similarity and encoding operations without exposing internal details.

— **Space Updater** : an event-consuming component responsible for updating the vector basis. It uses a buffer manager to aggregate incoming signals and triggers periodic or threshold-based updates. All update operations go exclusively through the *NLP Service*.

3.3.3 LLM Component

This figure illustrates the integration of the LLM service into the scoring pipeline, along with its role in semantic enrichment and the evaluation of candidate links.

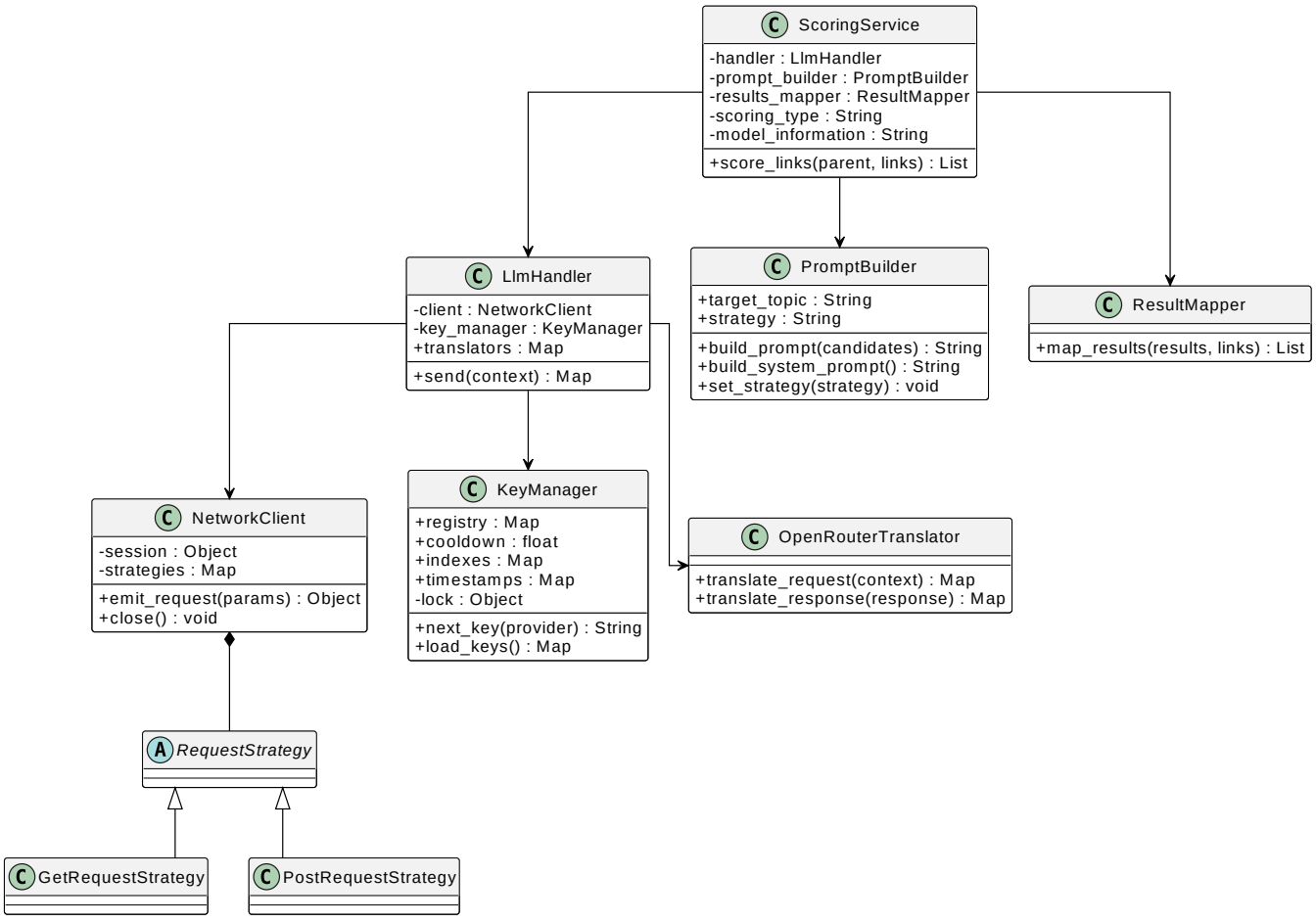

Figure 3.9: – Integration of the LLM service into the scoring architecture

The LLM system rests on an *LLM Handler*, which constitutes the central access point to the system's language models and can be used by different components to execute LLM requests uniformly.

The *LLM Handler* relies on three clearly separated sub-components :

- **Translators** : responsible for serializing the abstract context into the specific formats of each provider (e.g., OpenRouter), and for deserializing responses according to each API's own schema.
- **Key Manager** : manages the rotation and availability of API keys, applying cool-down policies. It exposes a `next_key(provider)` interface for dynamically obtaining a valid key.
- **Network Client** : the infrastructure layer responsible for the asynchronous, concurrent execution of HTTP requests.

This design establishes a strict separation between orchestration (LLM Handler), transport (Network Client), provider adaptation (Translators), and application logic (Scoring Service), thereby ensuring strong extensibility and isolation of external dependencies.

3.4 Sequence Diagrams

3.4.1 Starting a Crawl Session

This sequence diagram describes the process of starting a crawl session, including component initialization and the setup of the execution pipelines.

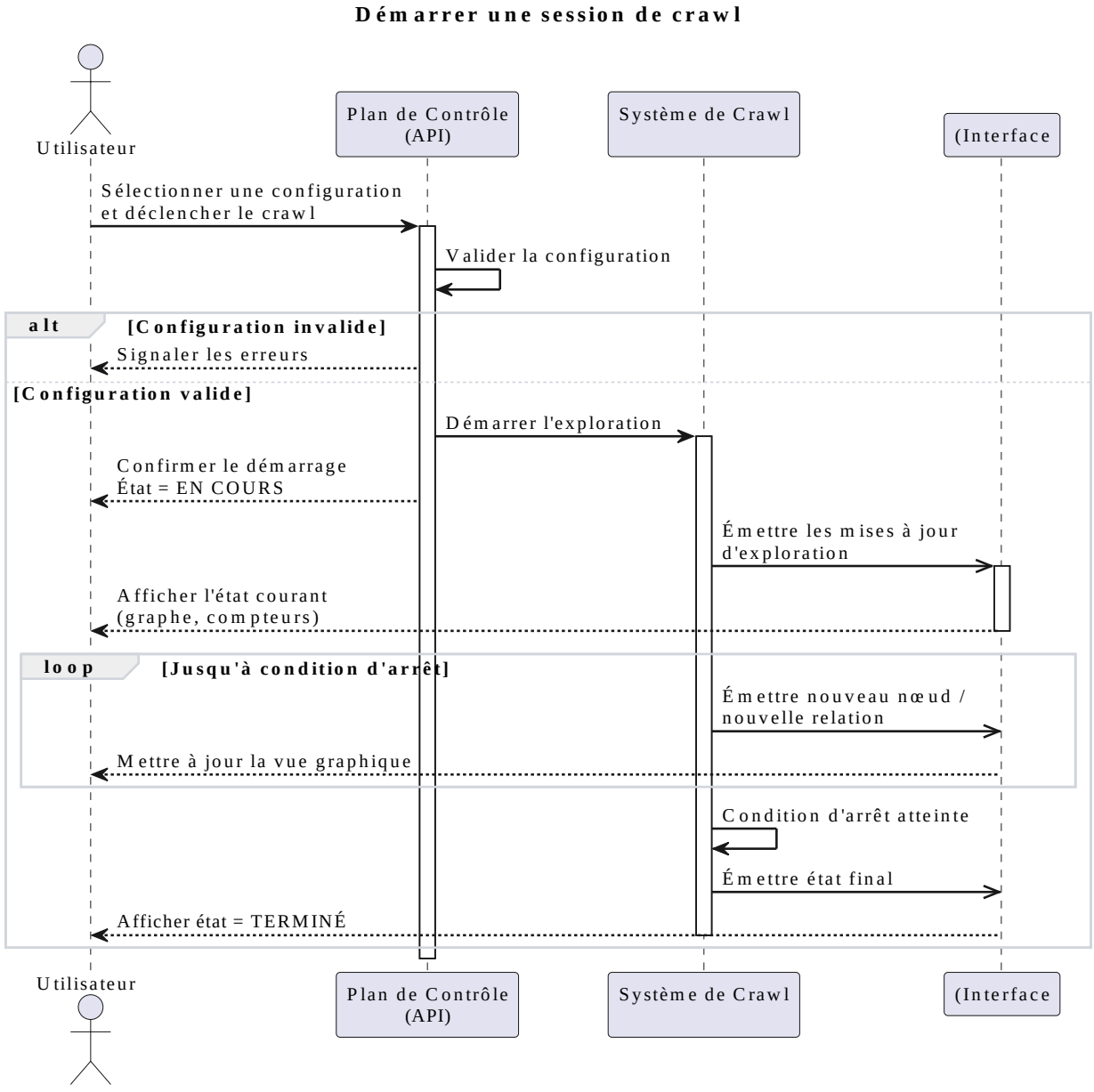

Figure 3.10: – Sequence for starting a crawl session

3.4.2 System Configuration

This sequence illustrates the process of configuring a blueprint, from parameter definition through validation and loading into the system.

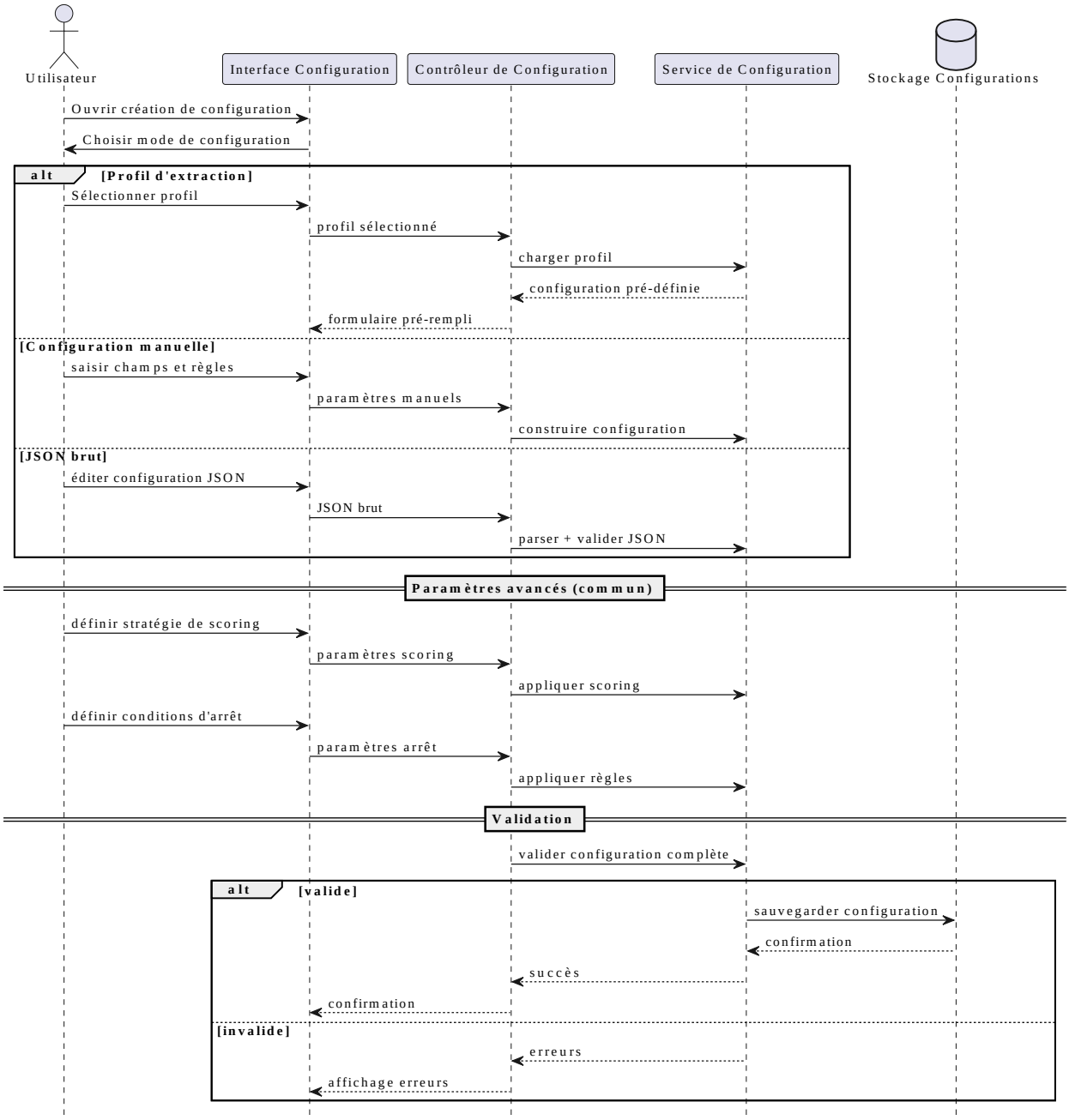

Figure 3.11: – Sequence for configuring a crawl blueprint

3.4.3 Inspection and Monitoring

This diagram describes the interaction flow enabling the inspection of a running or finished session, including consultation of events and results.

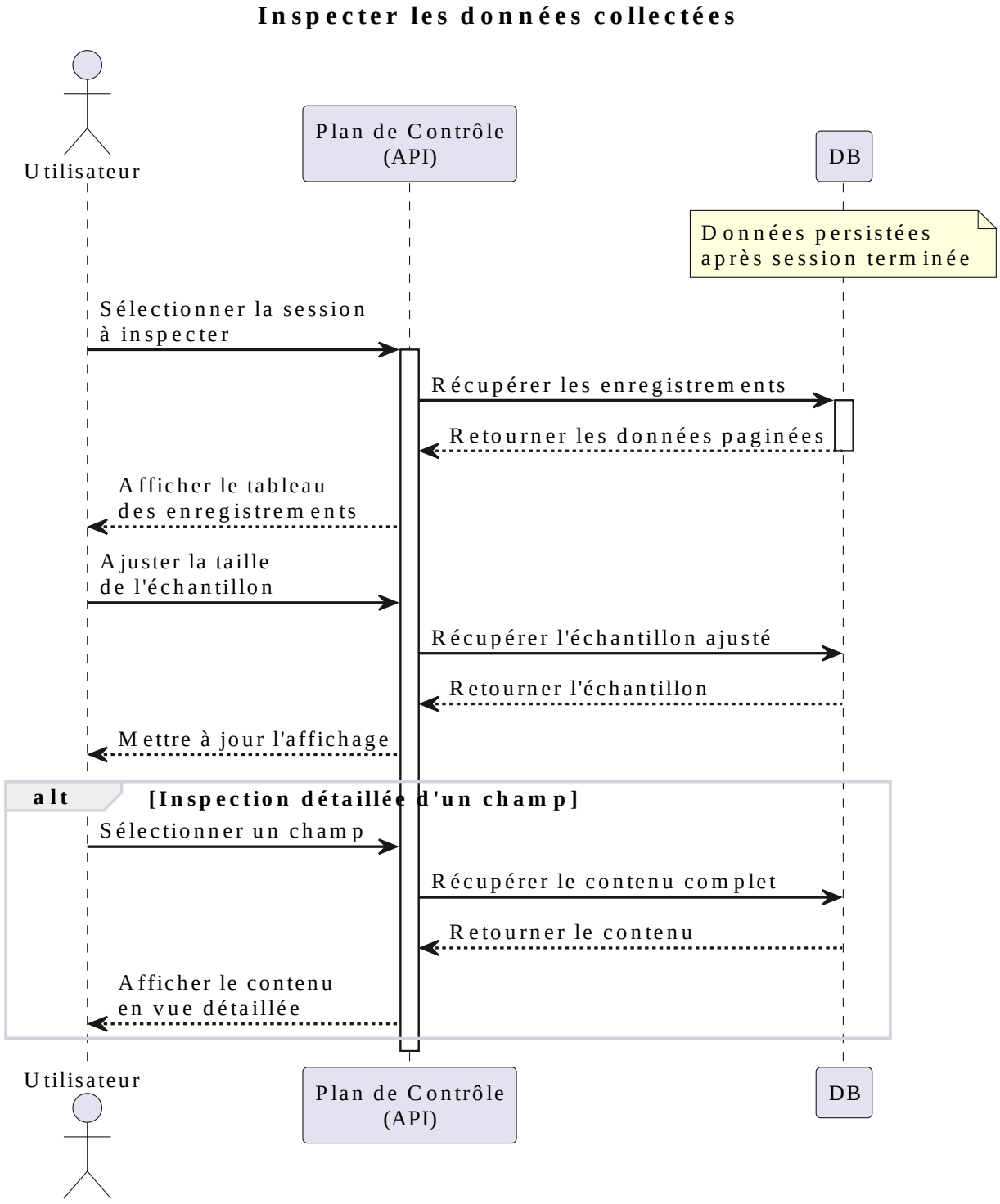

Figure 3.12: – Sequence for inspecting and observing a session

3.4.4 System Monitoring

This sequence represents the system's monitoring mechanism, based on the collection and aggregation of events produced during execution.

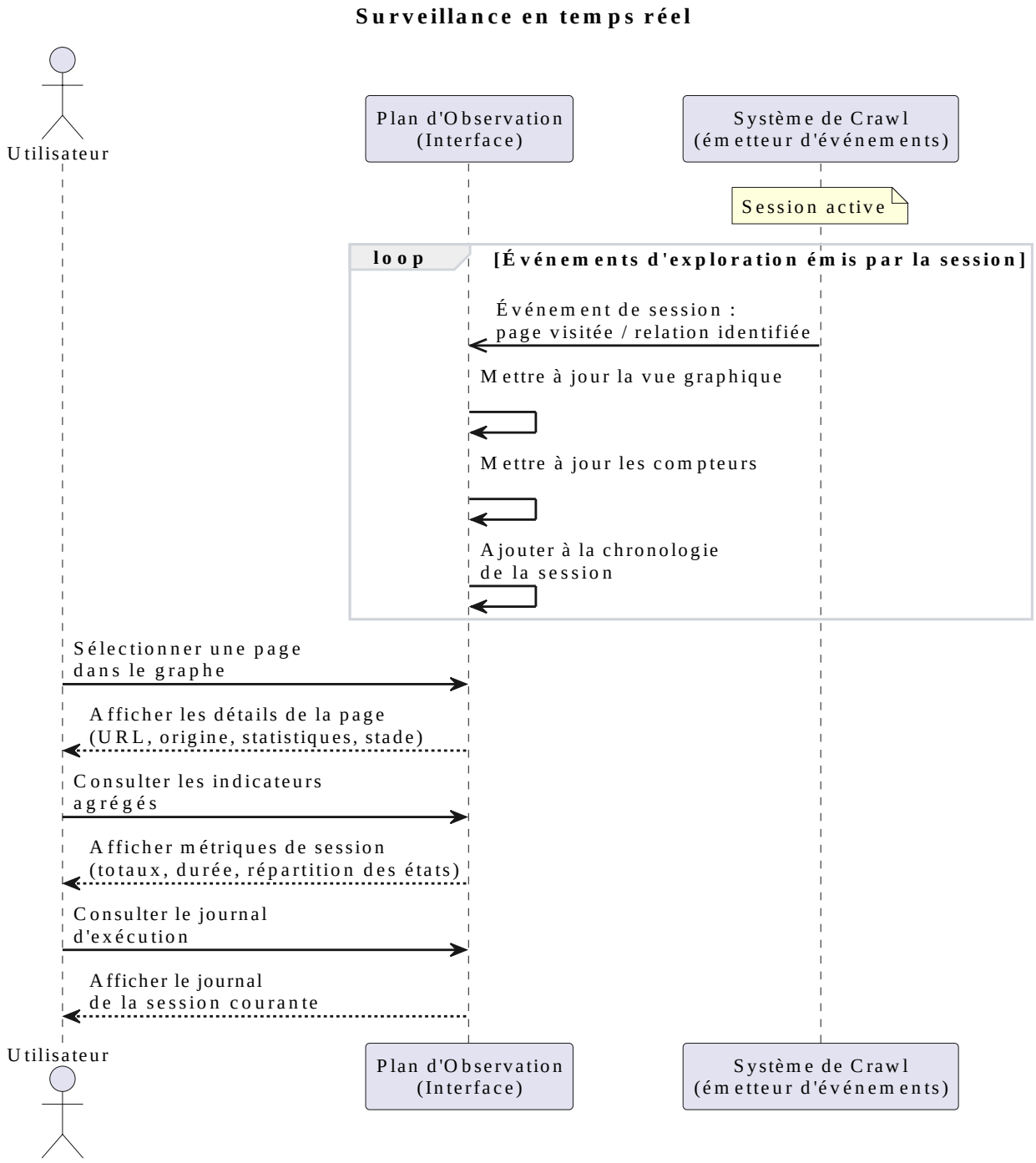

Figure 3.13: – Sequence for system monitoring

3.4.5 Stopping a Session

This sequence diagram illustrates the controlled shutdown process of a crawl session and the finalization of active pipelines.

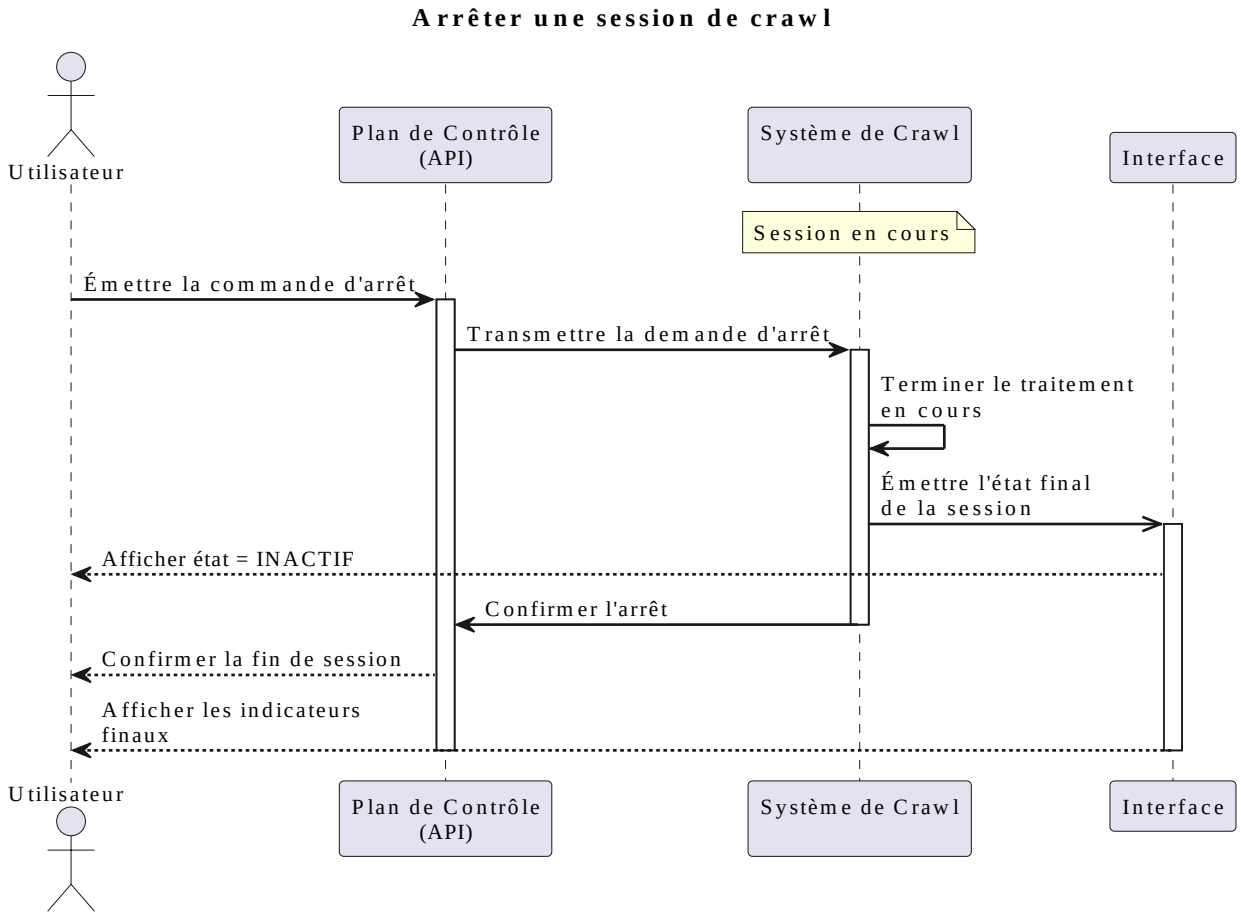

Figure 3.14: – Sequence for the controlled shutdown of a crawl session

Conclusion

This chapter presented CrawlViz's overall design along with its event-driven architecture based on specialized pipelines. The choices made ensure a clear separation of responsibilities, strong modularity, and full traceability of executions.

The proposed architecture thus provides an extensible and observable framework for focused crawling. The next chapter presents its concrete implementation, the technologies used, and the evaluation of the system.

4

Implementation

Introduction

This chapter presents the concrete implementation of CrawlViz through its technical implementation, its actual software architecture, and its user interfaces. It highlights the technology choices directly tied to the system's constraints : asynchrony, event-driven processing, and NLP/LLM service integration.

4.1 Environment and Technology Choices

4.1.1 Language and Concurrency Model

CrawlViz is developed in Python with the `asyncio` runtime. The choice of Python rests mainly on the maturity of its NLP and LLM ecosystem, notably `sentence-transformers` for semantic encoding and `aiohttp` for integrating LLM APIs. In addition, `asyncio`'s cooperative concurrency model is particularly well suited to a predominantly I/O-bound system such as web crawling, where network latency dominates processing time [9].

Python's ecosystem also makes it easy to integrate real-time communication via WebSocket. In the system, `FastAPI` exposes the crawler's REST control plane (start, stop, configuration),

while WebSockets carry the event stream to the interface [11].

4.1.2 Frontend Architecture

The user interface is built on React in order to support a continuous stream of real-time events (nodes, links, metrics). The `useReducer` model guarantees deterministic state transitions, ensuring the consistency of the graph even in the presence of frequent WebSocket-driven updates.

Table 4.1: – Main technologies used in CrawlViz

Technology	**Role**	**Justification**
Python / asyncio	Asynchronous concurrency	Suited to I/O-bound tasks
aiohttp	Asynchronous HTTP client	Non-blocking requests
lxml	HTML parsing	XPath/CSS compatible
Sentence-Transformers	Semantic encoding	Local NLP scoring
OpenRouter API	Access to LLM models	Multi-model interface
SQLite (WAL)	Local persistence	Lightweight and serverless
aiofiles	Disk logging	Non-blocking writes
FastAPI	REST control plane	Lightweight, fast API
React	User interface	Real-time visualization
WebSocket	Real-time communication	Instant updates

4.2 Presentation of the Interfaces

4.2.1 Execution and Control

The crawler's main control screen lets the user select a template from the list of available models, then launch execution via the *[Run Crawl]* button. Execution can also be interrupted at any time by clicking *[Stop]*.

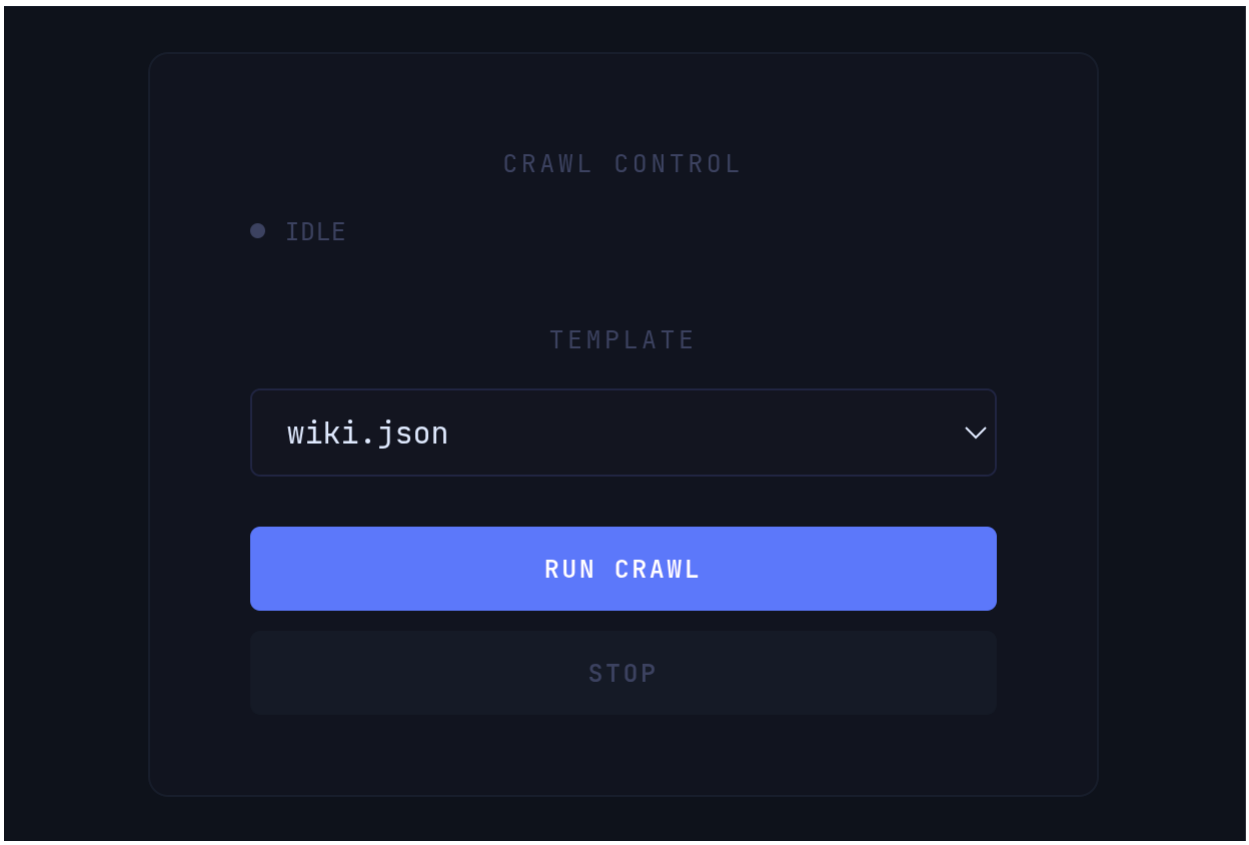

Figure 4.1: – Crawl execution control (starting and stopping a session).

4.2.2 Graph Visualization

This view is the system's main interface. It makes it possible to visualize the crawl's progress in real time. The user can observe the exploration graph, the color legend representing the different node states, and a side panel gathering global metrics, the event stream, and the replay controls.

Figure 4.2: – Global view of the exploration graph (seed : Diabetes).

During exploration, certain structures emerge naturally within the graph.

Hub A hub is a node with a high number of connections to other nodes. These nodes generally play a central role in the graph's structure. In this example, the *Diabetes* node constitutes a major hub.

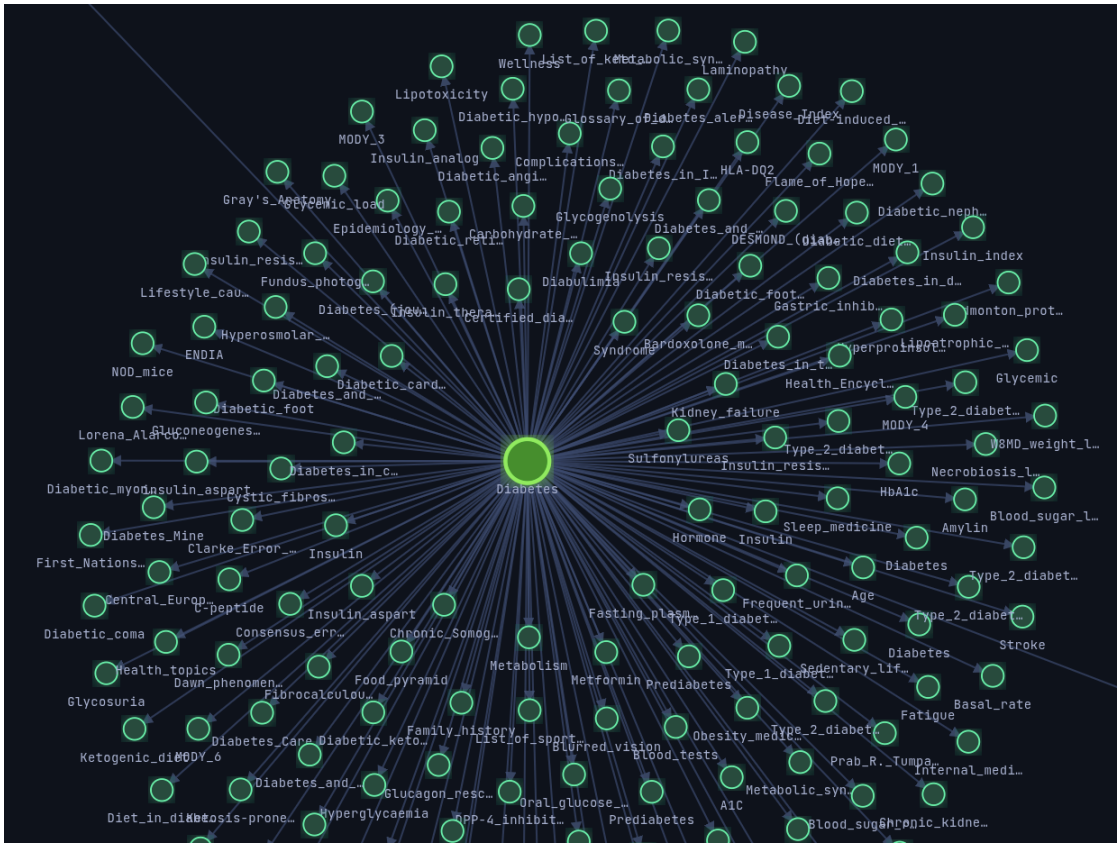

Figure 4.3: – Example of a hub centered on the topic of Diabetes.

Bridge Node A bridge node connects several regions or communities of the graph. Although it generally has few direct connections, it plays an important role in the network's overall connectivity.

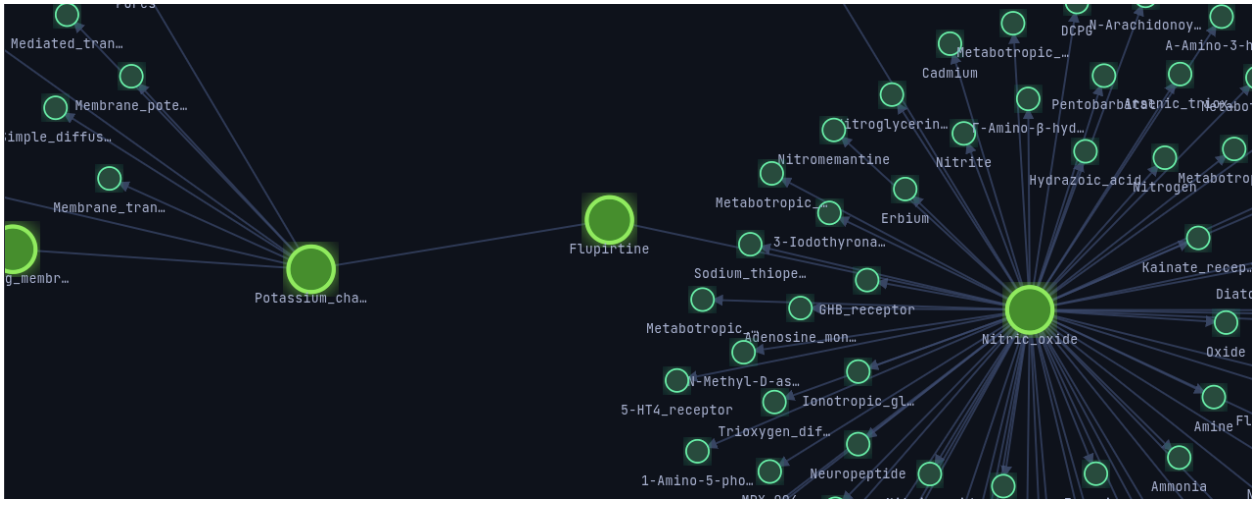

Figure 4.4: – Example of a bridge node connecting different areas of the graph.

Leaf A leaf node sits at the frontier of exploration. It does not yet have children in the graph and generally represents a node the crawler has not yet expanded.

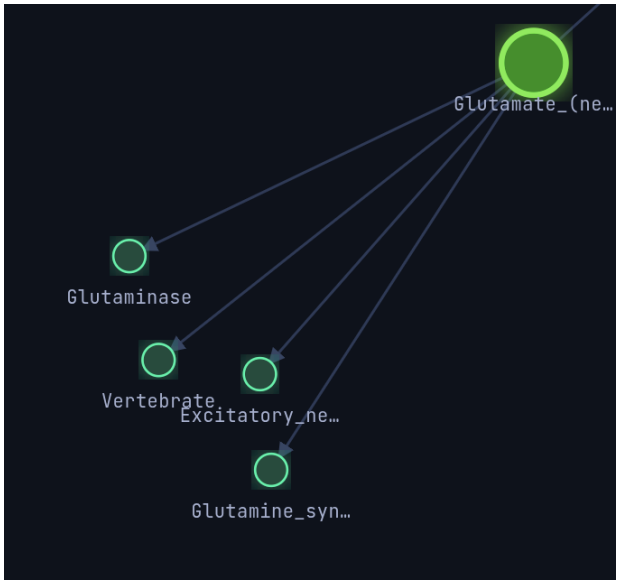

Figure 4.5: – Example of leaf nodes at the frontier of exploration.

The user can also navigate through the execution history using the time cursor or by directly selecting an event in the event stream. The graph is then rebuilt to reflect its state at the corresponding instant.

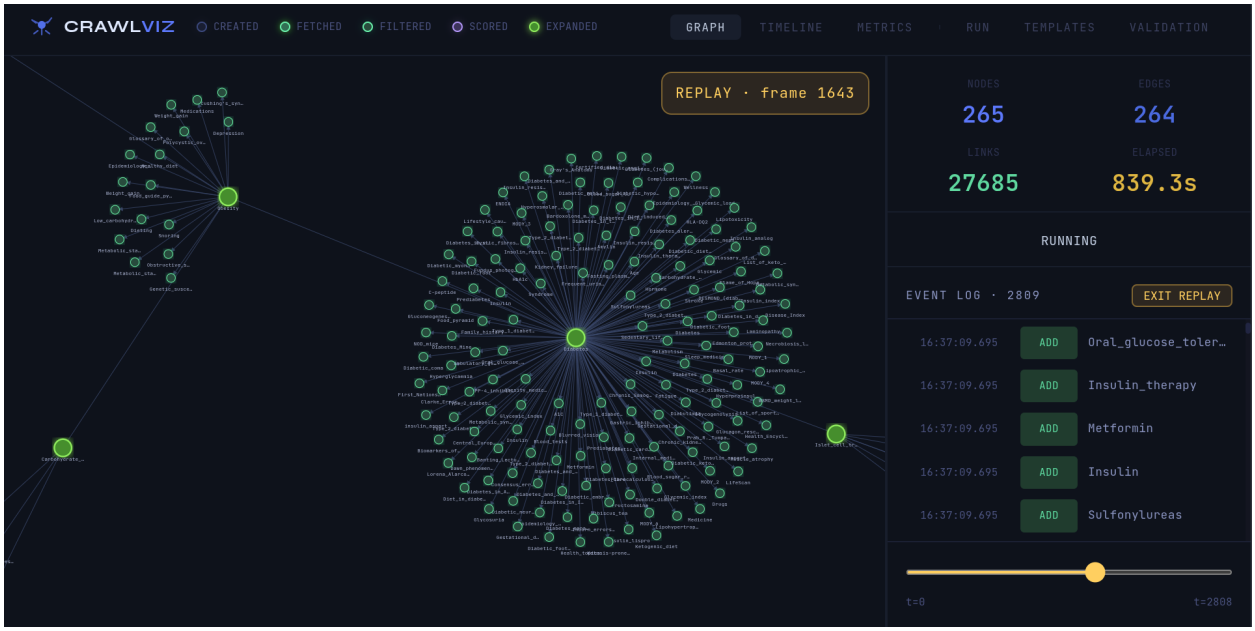

Figure 4.6: – Replay mode : temporal reconstruction of the graph.

4.2.3 Node Inspection

The user can click on a node to display its inspection panel. This panel presents the main metadata associated with that node's exploration and processing.

Figure 4.7: – Node inspection panel (example : ECG).

In this example, the node produced 201 accepted links and 70 filtered links, mainly due to the deduplication mechanisms. The panel also shows the value *Scored = 13*, corresponding to the final number of links retained after applying the NLP/LLM filtering-and-scoring cascade.

4.2.4 Timeline and Event Stream

This view presents the complete timeline of the system's main events during crawl execution.

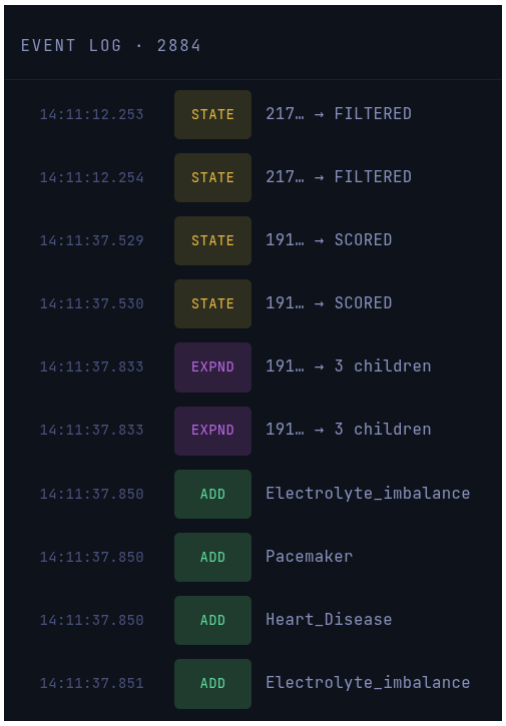

Figure 4.8: – Timeline of crawl events.

Here, one can observe the various node state transitions as well as the major events of the exploration process. In this example, the timeline notably highlights the addition of new nodes such as *Heart Disease* and *Electrolyte Imbalance*, along with their progression through the various processing stages.

4.2.5 Metrics

Global Metrics (Real Time)

This view presents the system's global metrics, including the total number of nodes explored, the edges between nodes, the links discovered, the extracted items, and the elapsed execution time.

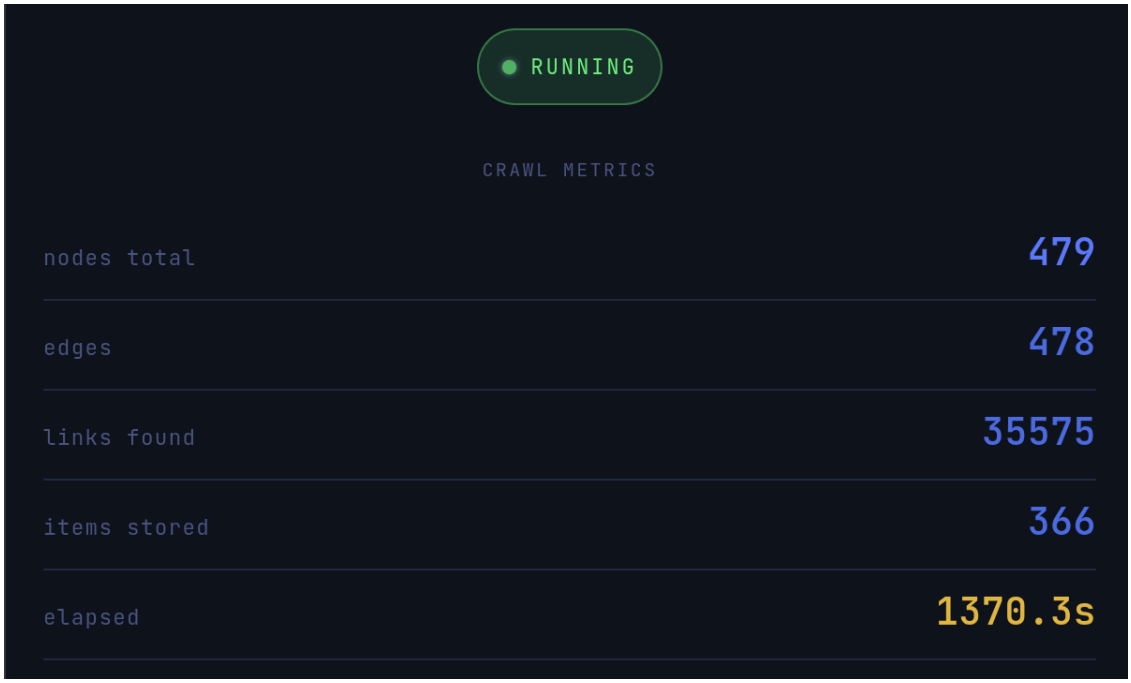

Figure 4.9: – Dashboard of global real-time metrics.

Global Metrics (Replay)

Statistics can also be consulted in replay mode, making it possible to follow the evolution of exploration over time and to dynamically analyze the crawler's behavior.

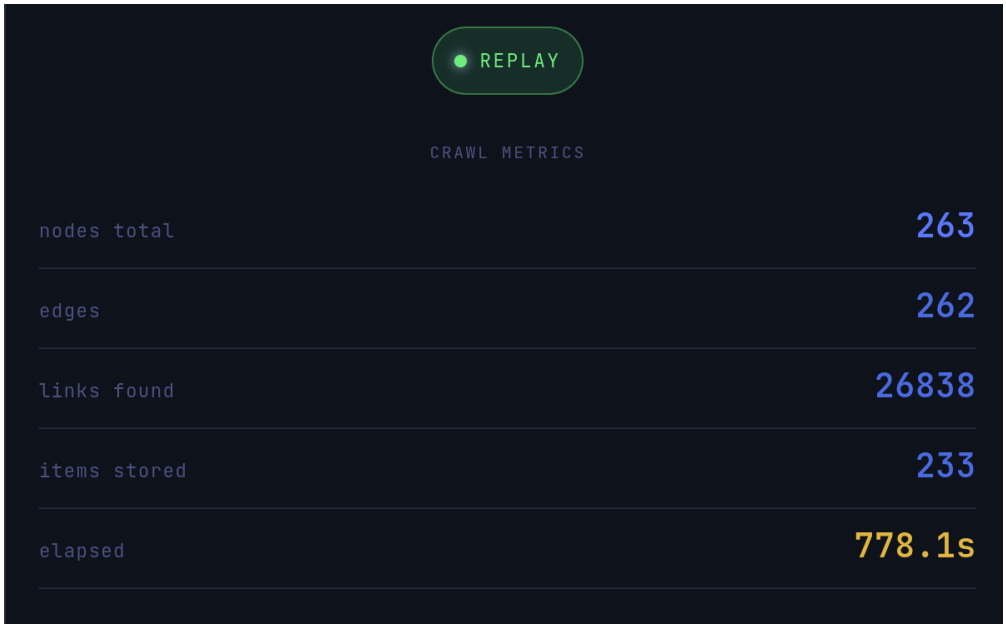

Figure 4.10: – Dashboard of global metrics in replay mode.

State Metrics

This view makes it possible to observe the distribution of node states, in real time and in replay mode. It facilitates the analysis of load distribution across the different processing pipelines.

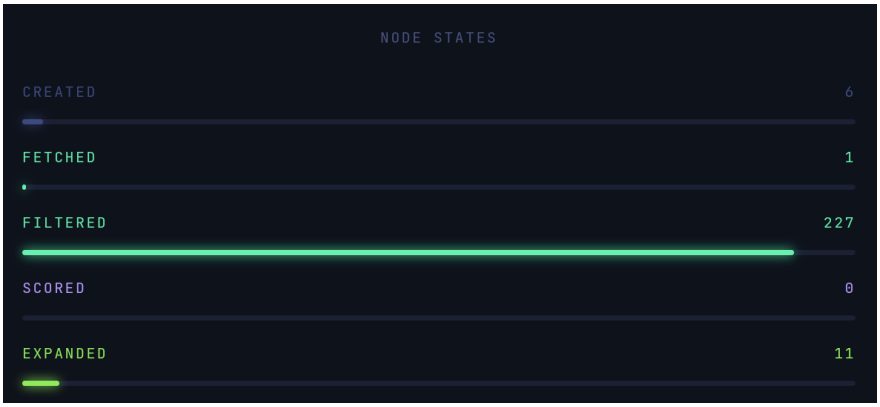

Figure 4.11: – Distribution of node states across the pipelines.

4.2.6 Data Validation

This view presents the interface for validating data collected during the crawl process. It allows a sample of the extracted data to be inspected. The user specifies the sample size, then launches inspection via the *Inspect* button. They can then browse through the different samples using the *Next* button.

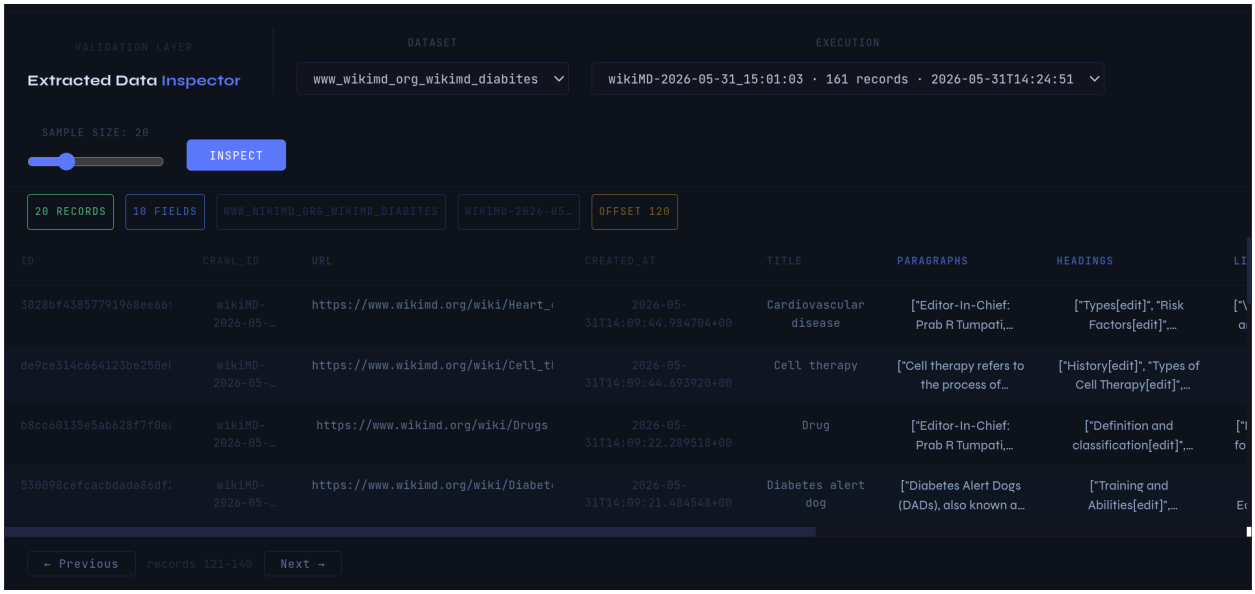

Figure 4.12: – Sampled view of the extracted data.

First, the user selects the dataset to analyze, generally defined by the combination of the domain's base URL and the associated blueprint.

Figure 4.13: – Dataset selection (domain and blueprint).

The user can then choose a specific crawl session to analyze. The system then displays the list of available sessions, along with their timestamp and the number of collected records.

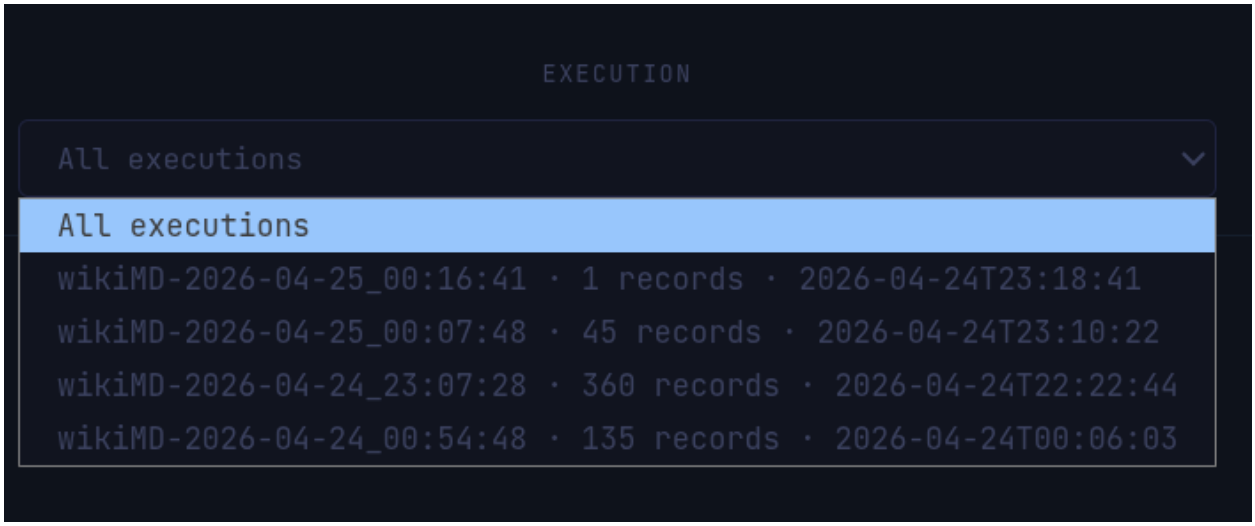

Figure 4.14: – Selecting a crawl session for inspection.

Finally, the user can explore the content in more detail by clicking on a table cell to display the underlying data associated with the selected record.

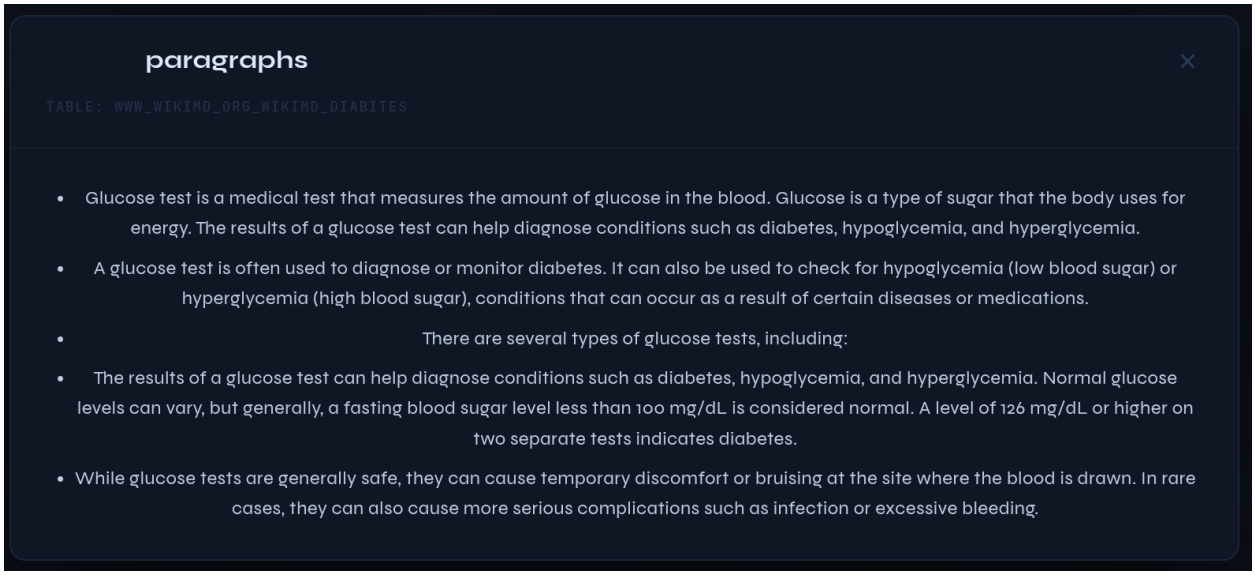

Figure 4.15: – Expanding an extracted field.

4.2.7 System Configuration

This view lets the user create and modify the system's *blueprints*. It offers two configuration modes.

Form mode allows structured data entry through dedicated fields.

In this interface, the user fills in the blueprint's metadata, such as the identifier, the definition of the target topic, and the *seeds*. New seeds can be added dynamically via the *[+ seed]* button.

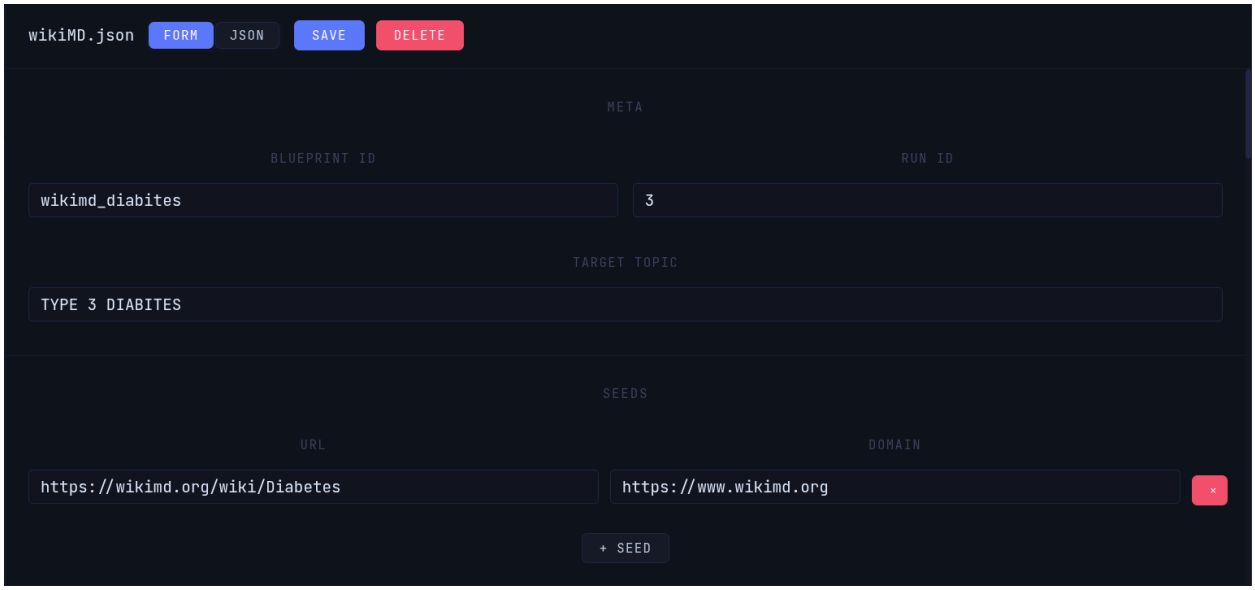

Figure 4.16: – Creating a blueprint (form mode).

The user can then add or remove domains. Each domain is defined by a name (or key), a base URL, and a link selector.

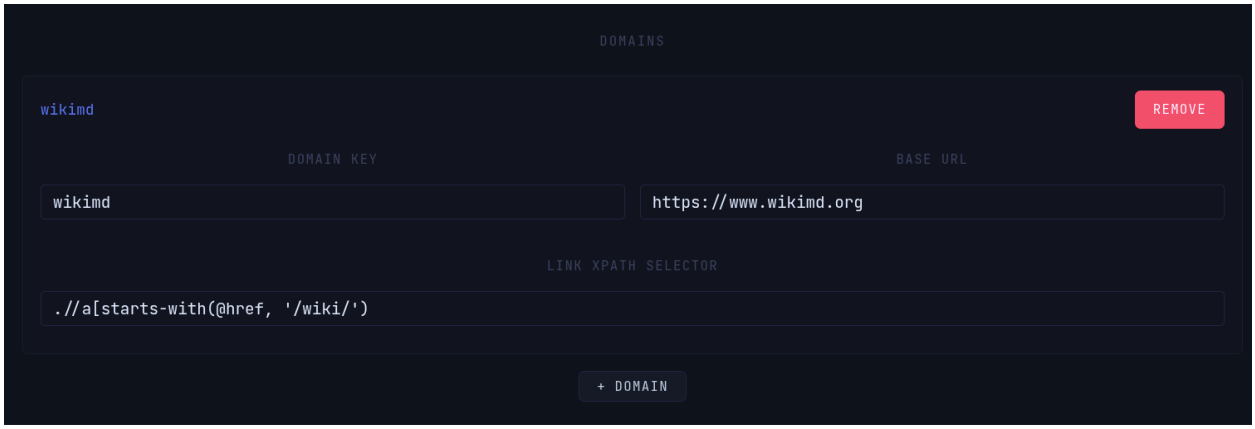

Figure 4.17: – Configuring the blueprint's domains (form mode).

The user then defines the scoring and expansion configurations. A strategy is chosen from a predefined list, along with the LLM backend used and the model identifier. Expansion parameters such as style, expansion count, and the associated LLM model are also configured.

Figure 4.18: – Configuring LLM scoring and expansion strategies.

The system then makes it possible to define the ETL logic. Two approaches are offered. In this mode, the user relies on a predefined profile by selecting fields, and the system automatically applies the associated rules.

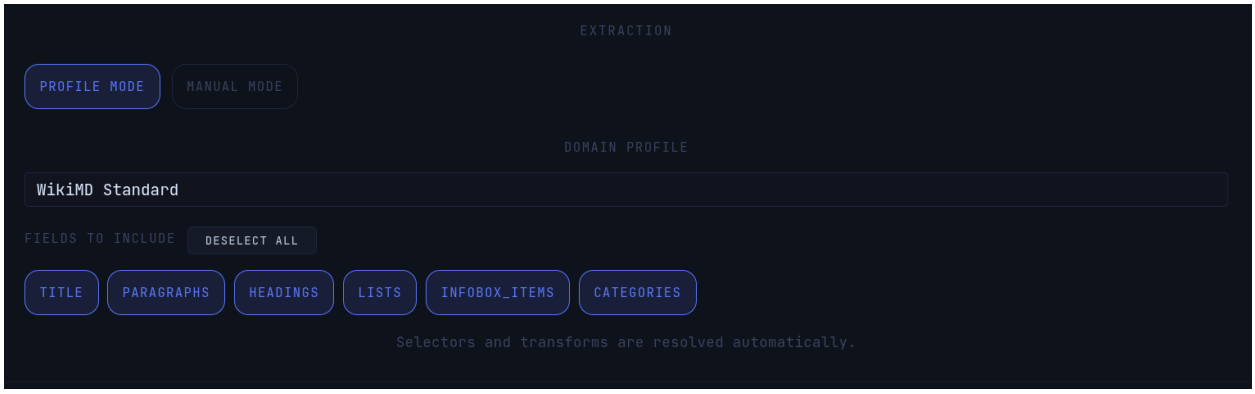

Figure 4.19: – Configuring the extraction profile (assisted mode).

The user can also choose manual mode, in which they define the fields themselves.

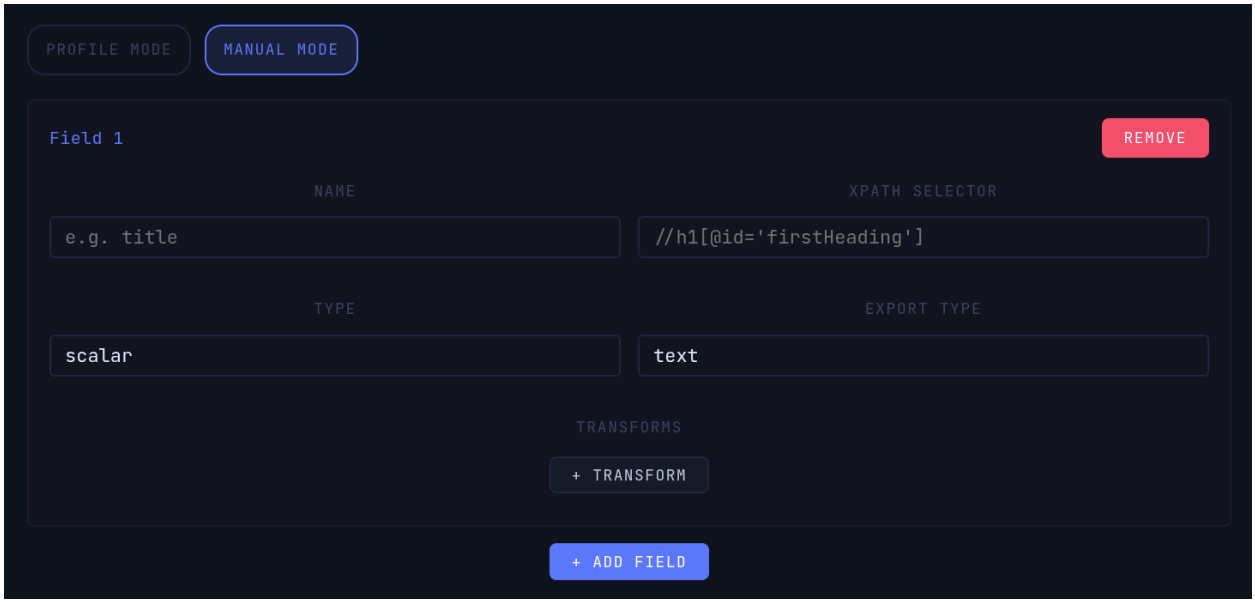

Figure 4.20: – Manual configuration of extraction fields.

In this mode, the user can define the nature of each field : either a scalar field or a list of elements.

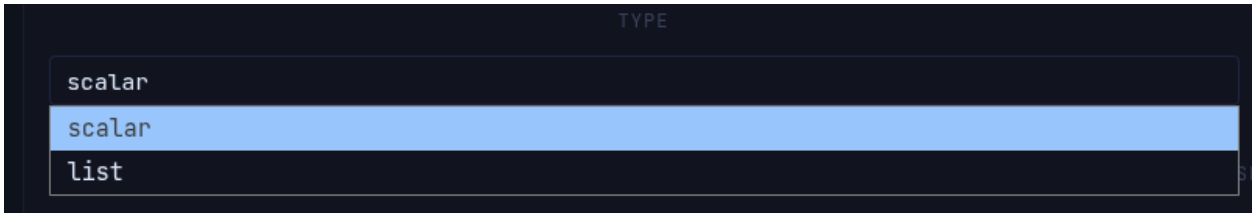

Figure 4.21: – Defining field types (scalar or list).

The user also defines the export type for the data.

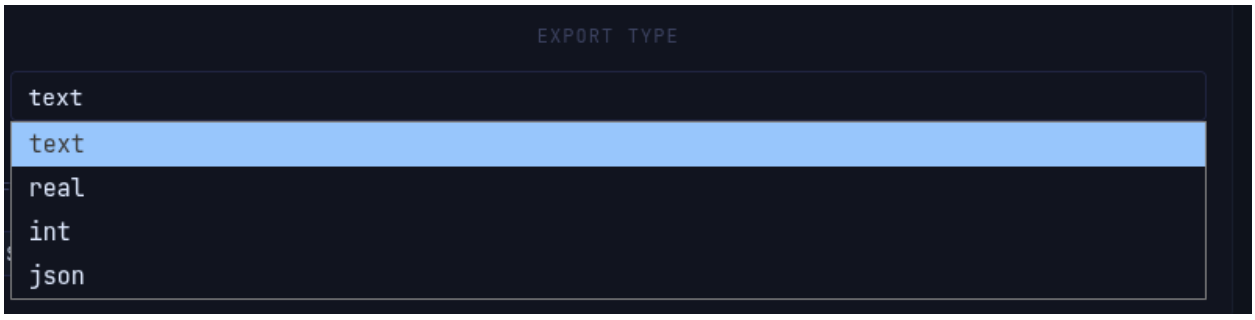

Figure 4.22: – Configuring the data export format.

The user can then define transformations applied to the extracted data. New transformations can be added via the *[+ transformation]* option, then selected from a predefined list.

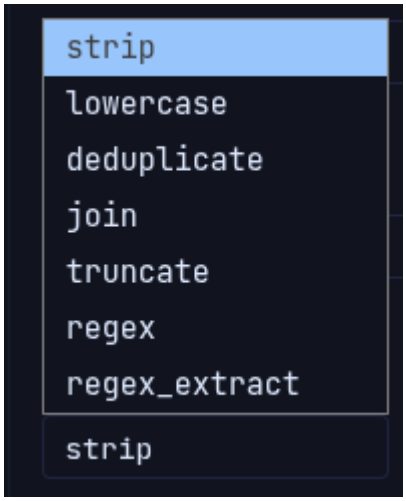

Figure 4.23: – Configuring transformations for the extracted data.

The user can also configure the crawl's stop conditions, in particular the maximum number of nodes and the maximum exploration depth.

Figure 4.24: – Configuring the crawl's stop conditions.

The system also allows manual editing of the JSON file. A base configuration is provided by the system, but the user can switch freely between the two modes.

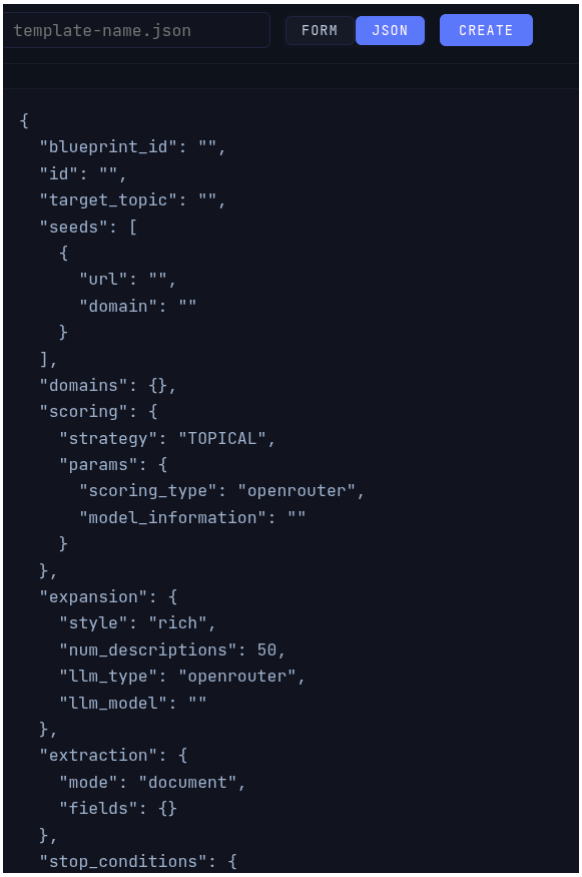

Figure 4.25: – Manual editing of the blueprint's JSON configuration.

Finally, when a blueprint is created, the system runs a full validation pass. If errors are found, a detailed list of the detected problems is displayed so they can be corrected.

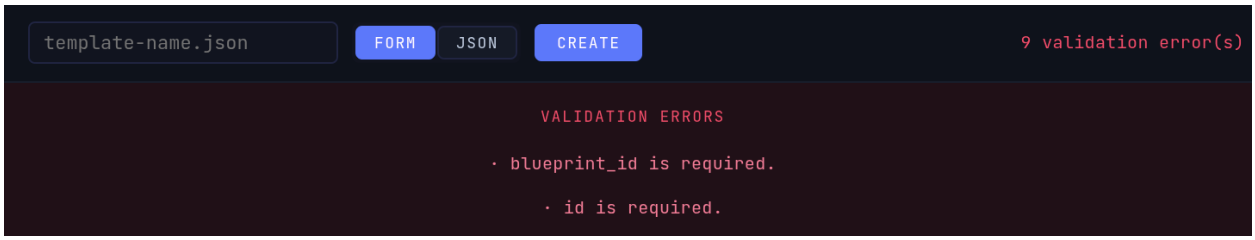

Figure 4.26: – Blueprint validation errors.

4.2.8 Observability and Logs

The logging system makes it possible to trace the crawler's execution. It notably displays the execution time as well as the debugging worker that emitted each event.

NLP Logs

NLP-type logs make it possible to validate the linguistic processing subsystem.

```
[13:11:17.157][DW1] [NLP] SCORE_EMITTED page=wikimd.org/wiki/Current_events nlp_score=0.058
[13:11:17.159][DW1] [NLP] SCORE_EMITTED page=wikimd.org/wiki/Glossaries nlp_score=0.0627
[13:11:17.166][DW1] [NLP] SCORE_EMITTED page=wikimd.org/wiki/Health_encyclopedia nlp_score=0.2136
[13:11:17.198][DW1] [NLP] SCORE_EMITTED page=wikimd.org/wiki/Fatigue nlp_score=0.2361
[13:11:17.233][DW1] [NLP] SCORE_EMITTED page=wikimd.org/wiki/Insulin nlp_score=0.3153
[13:11:17.255][DW0] [NLP] SCORE_EMITTED page=wikimd.org/wiki/Type_2_diabetes nlp_score=0.3475
[13:11:17.264][DW0] [NLP] SCORE_EMITTED page=wikimd.org/wiki/Glycemic_index nlp_score=0.4784
```

Figure 4.27: – NLP logs.

In this example, one can observe the NLP scores assigned to different pages according to their relevance to the target topic (for example, diabetes) :

- Navigation-type pages (*current events*) : low score (0.058).
- Glossaries : slightly higher score (0.0627).
- Encyclopedic health pages : medium-to-low relation (0.21).
- Strongly relevant elements, such as *glycemic index* : high score (0.47).

It can also be observed that *type2_ diabetes* obtains a lower score than *glycemic index*, because the score also incorporates exploration metrics, in particular novelty and diversity indicators.

Node-State Logs

The node states shown in the user interface are also traced in the logs. Here, one can observe the full lifecycle of a node, from its creation through to its final processing.

```
[2026-05-31 13:10:19.253][W0] [Node] id=0  → CREATED    page=wikimd.org/wiki/Diabetes depth=0
[2026-05-31 13:10:20.549][W0] [Node] id=0  → FETCHED
[2026-05-31 13:10:20.598][W0] [Node] id=0  → FILTERED   links=180 items=1
[2026-05-31 13:10:20.600][W0] [Node] id=0  → TRANSFORMED items=1
[2026-05-31 13:11:34.437][W0] [Node] id=0  → SCORED     links=69
[2026-05-31 13:11:34.444][W0] [Node] id=0  → EXPANDED   children=32
```

Figure 4.28: – State logs

LLM Logs

This section presents the logs related to the LLM as well as to the network (NET), illustrating in particular the full lifecycle of an interaction with the model.

```
[12:48:16.724][DW1] [LLM] PROMPT type=openrouter model=gpt-oss... len=1420 context=Scoring
[12:48:16.724][DW0] [LLM] DISPATCHED type=openrouter model=openai/gpt-oss-120b:free
[12:48:16.725][DW1] [NET] CREATED method=POST url=https://openrouter.ai/api... auth=True
[12:48:16.725][DW0] [NET] DISPATCHED strategy=Post
[12:48:24.086][DW0] [LLM] RESPONSE ok=True latency_ms=7917.2
[12:48:24.086][DW1] [NET] RESPONSE status=200 size=1981B latency_ms=7916.4
[12:48:24.090][DW1] [LLM] PARSED keys=['results']
[12:48:24.091][DW0] [PRIORITY] START
[12:48:24.091][DW1] [PRIORITY] DONE output=32
```

Figure 4.29: – Pipeline logs (LLM → network).

At the start of the process, the prompt is built using the selected model, the backend used, and its size. This prompt is then passed to the network client, which converts it into an HTTP request. The response is then received and parsed, after which the system computes the priority scores and the number of results generated.

Expansion Logs

This section concerns the logs associated with the generation of expansions.

```
[13:53:44.143][DW1] [EXP] seed=Characterized by brain insulin signaling deficits leading to…
[13:53:44.144][DW1] [EXP] seed=Often referred to as "brain diabetes" due to impaired cerebr…
[13:53:44.144][DW1] [EXP] seed=Hypothesized to involve accumulation of amyloid-beta interfe…
[13:53:44.145][DW1] [EXP] seed=Associated with chronic inflammation and oxidative stress wi…
[13:53:44.145][DW1] [EXP] seed=Linked to peripheral metabolic syndrome factors such as obes…
[13:53:44.146][DW1] [EXP] seed=Diagnosed through biomarkers like reduced cerebrospinal flui…
```

Figure 4.30: – LLM expansion-generation logs.

Here, one can observe a subset of the expansions generated, for example for the topic *type 3 diabetes*.

4.3 Practical Evaluation

4.3.1 Protocol

Table 4.2: – Experimental configuration of the reference run

Parameter	**Value**
Domain	`wikimd.org` (medical encyclopedia)
Starting seed	`/wiki/Diabetes` (TYPE 2 DIABETES)
Maximum number of nodes	600
Maximum depth	10 levels
Maximum time	1200 s
Priority strategy	AGGRESSIVE
Scoring strategy	TOPICAL
Number of expansions (LLM)	50

Use-case objective. This protocol aims to evaluate the system's ability to perform focused exploration in a dense web space while maintaining strong topical coherence. It illustrates the type of collection CrawlViz is designed to optimize.

Execution constraints and politeness. In order to respect the Web's *politeness* constraints and avoid overloading target servers, the system applies a rate limit fixed at 1 request per second per worker (2 workers in this experiment). This constraint interacts with the retry strategy based on *exponential backoff*, introducing a secondary rate-regulation mechanism. In practice, a significant share of execution time is thus spent in controlled waiting phases rather than in active processing.

4.3.2 Results of the Reference Run

Table 4.3: – Results of the reference run on the Diabetes topic

Metric	**Value**
Nodes explored	539
Links identified	50 828
Total crawl duration	1200 s

In order to analyze the system's performance, we decompose the total crawl duration into two components : the time induced by the politeness constraints and the effective execution time of the system.

Let :

— T_{total} be the total measured duration,

— N be the number of nodes explored,

— w be the number of workers,

— r be the rate imposed per worker (in requests/second).

The theoretical time imposed by the rate limit is given by :

$$T_{\text{rate}} = \frac{N}{w \cdot r} \tag{4.1}$$

The system's effective time is then :

$$T_{\text{eff}} = T_{\text{total}} - T_{\text{rate}} \tag{4.2}$$

Numerically :

$$T_{\text{rate}} = \frac{539}{2 \cdot 1} = 269.5 \text{ s} \tag{4.3}$$

$$T_{\text{eff}} = 1200 - 269.5 = 930.5 \text{ s} \tag{4.4}$$

Thus, about 22.5% of the total time is devoted to politeness constraints, while 77.5% corresponds to the system's internal processing (fetching, NLP/LLM scoring, and event orchestration).

Of the 50 828 links identified during exploration, only 539 nodes were ultimately retained for exploration, i.e., approximately 1% of the initial volume. This result illustrates the scoring

pipeline's ability to substantially reduce the search space while retaining the resources most relevant to the target topic.

Qualitative analysis of the resulting graph shows a clear structuring into thematic clusters corresponding to the main sub-domains of diabetes : complications (neuropathy, retinopathy, nephropathy), treatments (insulin, metformin, semaglutide), epidemiology, and associated pathologies (obesity, metabolic syndrome). Certain nodes play a bridging role, such as "Glycemic Index" and "HbA1c," ensuring connectivity between several thematic sub-groups, reflecting a structure close to the actual organization of the medical domain.

Conclusion

This chapter presented the concrete implementation of CrawlViz through the technologies adopted, the main interfaces developed, and the practical evaluation of the system. The results obtained show the crawler's ability to perform focused exploration while maintaining strong topical coherence and full observability of the execution process.

The analysis of the case study highlights the effectiveness of the semantic-selection mechanism, which substantially reduces the exploration space while retaining the most relevant resources. Taken together, these results validate the architectural and algorithmic choices presented in the previous chapters.

General Conclusion

This work presents the design and implementation of *CrawlViz*, a semantically-guided crawling system based on a scoring pipeline. The goal is to go beyond the limitations of classical crawlers [1] by prioritizing links according to their relevance, in order to improve the trade-off between exploration, cost, and quality.

The problem addressed is the exploration of a massive, noisy, and heterogeneous web. The challenge is to limit irrelevant exploration while retaining good coverage of the areas of interest.

The architecture rests on an asynchronous, modular system inspired by distributed architectures [9]. It combines an asynchronous crawler, a scoring pipeline based on language models, a prioritization system, and a structured extraction module. This separation improves relevance and reduces exploration noise.

Two main limitations emerge. The first is the dependency on language models, whose latency or unavailability slows down the system despite the tolerance mechanisms in place. The second is the sensitivity of the extractors to non-standardized HTML structures.

On the technical side, the project puts in place an asynchronous system integrating crawling, scoring, and prioritization with a clear separation of concerns. On the personal side, it strengthens a systemic approach to software design and to managing the trade-off between performance and relevance.

Future directions include the addition of JavaScript rendering engines, hierarchical extraction inspired by Scrapy, more robust extractors, and distributed parallelization to improve scalability.

Finally, the results on a diabetes-related case study show that CrawlViz builds coherent graphs from a single seed. It substantially reduces the explored space. The comparison with unfocused approaches confirms the value of semantic prioritization.

Bibliographie

[1] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[2] C. Olston and M. Najork, "Web Crawling," *Foundations and Trends in Information Retrieval*, vol. 4, no. 3, pp. 175–246, 2010.

[3] R. Mitchell, *Web Scraping with Python : Collecting More Data from the Modern Web*, 2nd ed. O'Reilly Media, 2018.

[4] S. Chakrabarti, M. van den Berg, and B. Dom, "Focused crawling : a new approach to topic-specific Web resource discovery," *Computer Networks*, vol. 31, no. 11–16, pp. 1623–1640, 1999.

[5] F. Menczer, G. Pant, and P. Srinivasan, "Topical web crawlers : Evaluating adaptive algorithms," *ACM Transactions on Internet Technology*, vol. 4, no. 4, pp. 378–419, 2004.

[6] [Reference to be completed by the author – the original report cites this key for the general claim that a pre-existing LLM can be used to judge topical relevance without task-specific training ; the exact source (paper, technical report, or blog post) used by the original bibliography could not be established from the material provided for this translation and should be filled in from the original report's `.bib` file.]

[7] N. Reimers and I. Gurevych, "Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[8] M. Brooker, "Exponential Backoff and Jitter," *AWS Architecture Blog*, Amazon Web Services, 2015.

[9] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[10] M. Fowler, "Event Sourcing," *martinfowler.com*.

[11] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD dissertation, University of California, Irvine, 2000.